ACCELERATING GRAPHICS RENDERING ON RISC-V GPUS

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Electrical Engineering

by

Joshua Simpson

June 2022

© 2022
Joshua Simpson
ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE:	Accelerating Graphics Rendering on RISC-V GPUs
AUTHOR:	Joshua Simpson
DATE SUBMITTED:	June 2022
COMMITTEE CHAIR:	Joseph Callenes-Sloan, Ph.D. Assistant Professor of Computer Engineering
COMMITTEE MEMBER:	Andrew Danowitz, Ph.D. Associate Professor of Computer Engineering
COMMITTEE MEMBER:	Maria Pantoja, Ph.D. Associate Professor of Computer Engineering

ABSTRACT

Accelerating Graphics Rendering on RISC-V GPUs

Joshua Simpson

Graphics Processing Units (GPUs) are commonly used to accelerate massively parallel workloads across a wide range of applications from machine learning to cryptocurrency mining. The original application for GPUs, however, was to accelerate graphics rendering which remains popular today through video gaming and video rendering. While GPUs began as fixed function hardware with minimal programmability, modern GPUs have adopted a design with many programmable cores and supporting fixed function hardware for rasterization, texture sampling, and render output tasks. This balance enables GPUs to be used for general purpose computing and still remain adept at graphics rendering. Previous work at the Georgia Institute of Technology has been done to implement a general purpose GPU (GPGPU) in the open source RISC-V ISA. The implementation features many programmable cores and texture sampling support. However, creating a truly modern GPU based on the RISC-V ISA requires the addition of fixed function hardware units for rasterization and render output tasks in order to meet the demands of current graphics APIs such as OpenGL or Vulkan. This thesis discusses the work done by students at the Georgia Institute of Technology and California Polytechnic State University SLO to accelerate graphics rendering on RISC-V GPUs including the specific contributions made to implement and connect fixed function graphics hardware for the render output unit (ROP) to the programmable cores in a RISC-V GPU. This thesis also explores the performance and area cost of different hardware configurations within the implemented GPU.

ACKNOWLEDGMENTS

Thanks to:

- My advisors, Professors Joseph Callenes-Sloan and Andrew Danowitz for their guidance, advice, and time throughout this project.
- My other committee member, Professor Maria Pantoja, for her expertise and time.
- My family for all their support and encouragement. If it wasn't for my dad, I would not have developed my love for computers!

TABLE OF CONTENTS

Page

LIST OF TABLES viii

LIST OF FIGURES ix

CHAPTER

1 Introduction 1

2 Background 3

2.1 RISC-V Instruction Set Architecture 3

2.2 Graphics Rendering Pipeline 4

2.3 GPU Architecture Overview 6

2.4 Vortex GPGPU 7

2.5 Related Work 11

2.5.1 RISC-V Extension to Support GPGPU/GPU 12

2.5.2 FPGA Based GPUs 12

3 Graphics Acceleration Hardware Design 14

3.1 Project Goal 14

3.2 Overview of New Vortex Architecture 15

3.3 Hardware Contributions 20

3.3.1 ROP Blending Unit 20

3.3.2 ROP Input Crossbar 22

3.3.3 ROP Cache Additions 23

3.4 Triangle Rendering Example 24

3.4.1 Geometry Stage 24

3.4.2 Rasterization Stage 26

3.4.3 Fragment Stage 26
4 Results and Analysis 28
4.1 Results 29
4.1.1 Varying Number of Cores, Warps and Threads 30
4.1.2 Software vs Hardware Rendering 30
4.1.3 Adding a Raster and ROP Cache 33
4.1.4 Varying number of Raster/ROP Slices 36
4.1.5 Varying Raster Tile Size 37
4.1.6 Area usage 38
4.2 Analysis 41
4.2.1 Varying Number of Cores, Warps and Threads 41
4.2.2 Software vs Hardware Rendering 42
4.2.3 Adding a Raster and ROP Cache 43
4.2.4 Varying number of Raster/ROP Slices 44
4.2.5 Varying Raster Tile Size 44
4.2.6 Area usage 45
5 Conclusion 47
5.1 Future Work 48
BIBLIOGRAPHY 50
APPENDICES
A Results for Software vs Hardware Rendering (Continued) 53

LIST OF TABLES

Table Page

4.1 Benchmark descriptions in terms of draw calls, number of vertices, number of primitives, number of textures, depth/stencil test enable, and blending enable 29

LIST OF FIGURES

Figure		Page
2.1	Overview of the graphics rendering pipeline.	4
2.2	Overview of original Vortex GPGPU architecture with five stage pipeline, texture sampling, and scalable design [18].	9
2.3	Vortex GPGPU driver interface using Intel OPAE to interface FPGA with host CPU across PCIe bus [18].	10
2.4	Vortex GPGPU simulation options, including FPGA implementation, Verilator with OPAE (VLSIM), Verilator without OPAE (RTLSIM), and the C++ simulator (SIMX) [18].	11
3.1	Vortex implementation of graphics pipeline divided into host CPU and GPU sections.	16
3.2	New Vortex GPU architecture with raster and ROP units added at the cluster level. Service blocks in each core direct requests to the raster and ROP units.	16
3.3	Architecture of Vortex raster unit with data structures, raster slices, and tile evaluators detailed.	18
3.4	Architecture of Vortex ROP unit showing the parallel depth/stencil test and blend unit inside each slice.	19
3.5	Design of blend unit inside ROP slice. Only one compute lane shown.	22
3.6	Overview of cache bypass design allowing ROP cache to be omitted in favor of directly accessing main memory.	23
3.7	Vertex shader code example. Host CPU executes this program on a each vertex.	25
3.8	Visual example displaying results of geometry stage in graphics pipeline. Green arrow represents vertex shader.	25
3.9	Visual example displaying results of rasterization stage in graphics pipeline. Each blue arrow represents a process within the raster unit hardware.	26

3.10 Fragment shader code example. Each thread in Vortex's programmable cores executes this program on a different fragment. 27

3.11 Visual example displaying results of fragment stage in graphics pipeline. The green arrow represents the fragment shader, the blue arrow represents a process within the ROP unit hardware. 27

4.1 Sample renders of each benchmark (vase, filmtv, skybox, coverflow, evilskull, polybump, tekkaman, carnival) [6] 28

4.2 Instructions Per Clock (IPC) vs Number of Cores for each of the eight benchmarks. As the number of cores is doubled, IPC scales by an average factor of 1.6 times. 31

4.3 Instructions Per Clock (IPC) vs Number of Warps Per Core for each of the eight benchmarks. As the number of warps is doubled, IPC scales by an average factor of 1.45 times. Verilator failed to compile GPU when number of warps was set to one. 31

4.4 Instructions Per Clock (IPC) vs Number of Threads Per Warp for each of the eight benchmarks. As the number of threads is doubled, IPC scales by an average factor of 1.7 times. Verilator failed to compile GPU when number of threads was set to one. 32

4.5 Total number of clock cycles to complete FilmTV benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.5 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 33

4.6 Total number of clock cycles to complete Tekkaman benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 1.1 to 4.5 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 34

4.7 Total number of clock cycles to complete Carnival benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 19 to 38 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 34

4.8 Instructions Per Clock (IPC) vs Type of Cache Configuration for each of the eight benchmarks. All results within the same benchmark stayed within 3% of each other. 35

4.9 Instructions Per Clock (IPC) vs Type of Cache Configuration for various raster tile sizes using the Carnival benchmark. Above an 8x8 tile size, RCACHE + OCACHE configuration consistently performs faster, but by less than 1% across the board. 36

4.10 Instructions Per Clock (IPC) vs Number of Raster/ROP Slices for various core configurations using the Vase benchmark. No significant difference observed between different numbers of slices. 37

4.11 Instructions Per Clock (IPC) vs Raster Tile Size using the Carnival benchmark. Tile size of 128x128 perform best while a tile size of 8x8 performs nearly 10% slower. 38

4.12 Area cost of Vortex GPU measured in Combinational ALUTs utilized per unit. A single core, four warp, four thread configuration was used. 39

4.13 Area cost of Vortex GPU measured in Logic Registers utilized per unit. A single core, four warp, four thread configuration was used. . 39

4.14 Area cost of Vortex GPU measured in DSP Slices utilized per unit. A single core, four warp, four thread configuration was used. 40

A.1 Total number of clock cycles to complete Vase benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.8 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 53

A.2 Total number of clock cycles to complete Skybox benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5.1 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 54

A.3 Total number of clock cycles to complete Coverflow benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5.1 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 54

A.4 Total number of clock cycles to complete Evilskull benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 55

A.5 Total number of clock cycles to complete Polybump benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results. 55

Chapter 1

INTRODUCTION

While modern Graphics Processing Units (GPUs) have been used to accelerate machine learning [8], cryptocurrency mining [3], and many other massively parallel computations using languages such as NVIDIA's CUDA and OpenCL, the original application for GPUs that motivated their design was the acceleration of the 3D graphics pipeline. The GPU came into existence as a purely fixed function hardware unit to process each stage of the graphics rendering pipeline. This design morphed into a partially programmable accelerator when developers realized the expanded capabilities offered with vertex and pixel shaders [21]. There was still lacking the general purpose compute power that a modern GPU contains due to the highly specialized and optimized programmability of these shaders. Around the end of 2006, NVIDIA announced the G80 GPU architecture, introducing the concept of unified shaders. This architecture ushered in a new era for GPUs that still persists today: large numbers of fully programmable cores called stream processors (or CUDA/shader cores) [9]. These cores can be used to execute all of the shaders needed in the different stages of the graphics rendering pipeline (hence the name "unified shader") and can be used to run general purpose, parallel computing code.

Despite the push towards unified shaders and programmable cores, certain portions of the graphics pipeline have continued to remain fixed function due to performance, power, and area constraints. The latest NVIDIA GPU architecture, Ampere, contains a number of hardware blocks besides the programmable shader cores. These include texture units, render output units (ROPs), raster engines, and polymorph engines in terms of fixed-function hardware [10]. This GPU architecture also contains

tensor [10] cores and ray-tracing cores to accelerate specific workloads for graphics and non-graphics related tasks. Mobile GPU architectures feature similar components but with even stricter power and area budgets [16]. Efforts have been made to compute almost the entire graphics pipeline in software using the programmable cores of the GPU [13] but with little commercial success due to the degradation in performance. Deciding which parts of the graphics pipeline to do in software versus hardware is perhaps the most critical for low power, area constrained embedded and mobile devices. Finding an optimized balance between software and hardware depends on the application and the power/area budget allocated to the GPU.

This thesis discusses the work done by a team of students at both the Georgia Institute of Technology and California Polytechnic State University SLO to create an open source, modular GPU hardware implementation based on the RISC-V instruction set architecture (ISA) targeting a mobile and embedded device form factor. The design uses a mixture of software and fixed-function hardware to support the graphics rendering pipeline through the Vulkan and OpenGL-ES APIs. This work has been documented in a MICRO22 paper submission which will be used as a primary reference, but the design of the fixed function render output unit (ROP) will be discussed in more detail for this thesis. Additionally, the performance and area metrics of the design are analyzed with different hardware configurations to compare optimal design tradeoffs.

Chapter 2

BACKGROUND

2.1 RISC-V Instruction Set Architecture

The foundation of the work done for this thesis is based on the RISC-V instruction set architecture (ISA). The RISC-V ISA was developed at the University of California at Berkeley in 2010 and contains a number of unique features that set it apart from other instruction sets [7].

The original intention behind RISC-V was to be a tool for computer architecture research and education, but after widespread adoption it has begun to make significant headway in industry with its "free and open" architecture available for anyone to implement and modify as they see fit [20]. RISC-V was designed from the beginning to be an open-source ISA and is not just useful for simulation but also performance and energy optimized hardware implementations.

This flexibility is enabled by a number of specific features built into the RISC-V ISA. One such feature is the use of a small but separate base integer ISA while standard extensions are left optional for any additional requirements. This is coupled with extensive support for custom extensions to create hardware needed to accelerate a given task. The RISC-V ISA also supports both 32-bit and 64-bit implementations, support for the IEEE floating point standard, and multi-core and multiprocessor designs.

2.2 Graphics Rendering Pipeline

A primary task for GPUs is the rendering of 3D graphics to a screen. The graphics rendering pipeline, or just graphics pipeline, is the general name for the process that takes place on a GPU to turn 3D coordinates into 2D pixels on a screen [19]. These 3D coordinates might be the components of a simple primitive such as a point, line, or triangle. These primitives are the basic components that make up a larger 3D model. Computer Aided Design (CAD) software, video games, and animated movie scenes are common examples of 3D models. The graphics pipeline renders 3D models from a computer's memory to the 2D screens. Understanding the graphics pipeline is essential for understanding GPU architecture, since GPUs are designed specifically to process the stages of the graphics pipeline. There is no universal graphics pipeline, but a simple model with three stages is sufficient for describing the basic concept. These stages are commonly known as the Geometry, Rasterization, and Fragment Stage. An overview of the graphics pipeline is shown in Figure 2.1. Boxes colored yellow represent shaders, small programs designed to run on the programmable cores of a GPU. The uncolored boxes represent operations executed on the fixed function hardware of the GPU.

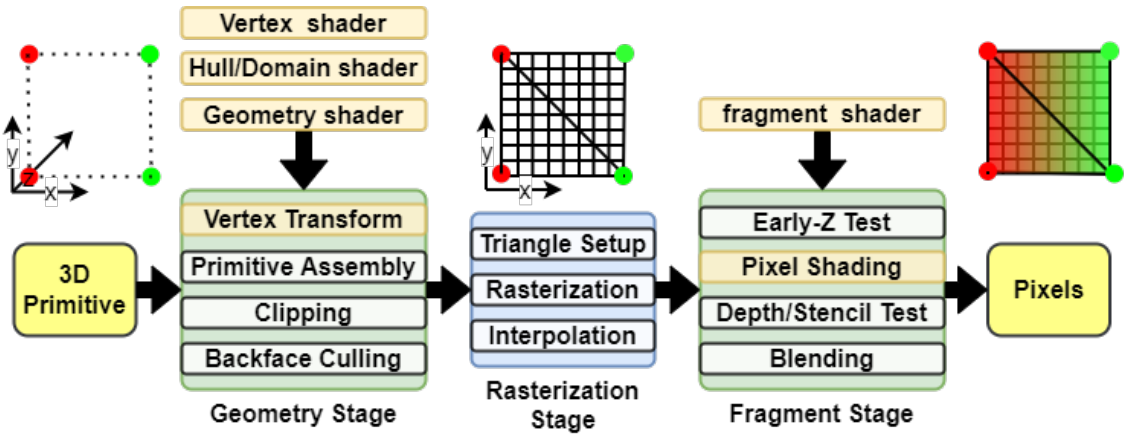


Figure 2.1: Overview of the graphics rendering pipeline.

At the beginning of the pipeline, a 3D object in computer memory is represented as a list of vertices organized into primitives such as points, lines, and triangles. These vertices can have attributes such as position, color, or lighting that form the desired primitive [17]. The vertices enter the Geometry stage of the pipeline which performs various tasks such as mapping input vertices to new 3D vertices (vertex shader), lighting calculations, grouping vertices into the correct primitives (primitive assembly), removing primitives that reside outside the rendering view (clipping), and transforming the 3D vertices in world space to the correct 2D locations in screen space.

The Geometry Stage outputs 2D primitives that are then passed to the Rasterization Stage. During rasterization, these primitives are turned into groups of fragments, a data type containing all the information needed to render a pixel [19]. The rasterization process takes a primitive (such as a triangle) and fills it in with squares (fragments) so that the geometry of the triangle can be displayed on a computer screen made up of square pixels. Fragments can also be filled in with interpolated color values based on the colors given to the vertices that created the rasterized primitive.

The fragments generated by rasterization are passed to the Fragment Stage of the graphics pipeline. The Fragment Stage performs texture sampling and calculates lighting on a per-fragment basis. This process computes the final color of a fragment. At this point in the graphics pipeline, the original 3D model is now represented as a 2D plane of colored squares (fragments). However, some objects from the original model might be in front of others in the scene with respect to the camera view and some might have transparent qualities. To generate the final image that is viewed on the computer's display, every pixel on the screen needs a final color assigned to it based on what objects are closest to the viewport of the camera. Using depth tests,

stencil tests, and color blending, each pixel is calculated from the attributes of all the fragments in the scene. Once this process is done, the final image is stored in the render target to be displayed on a screen.

2.3 GPU Architecture Overview

GPUs were originally designed around a fixed function implementation of the graphics pipeline. As the need for more flexibility in graphics workloads grew, the programmability of GPUs followed suit. With more applications using GPUs to accelerate parallel workloads, the concept of the General Purpose GPU (GPGPU) arose, driving the need for non-graphics related programmability at an architectural level.

Today, modern GPUs incorporate a mix of programmable shader cores with fixed function hardware for accelerating graphics specific tasks. An example of this is the NVIDIA Ampere architecture from 2020 [10].

This architecture features a maximum of 68 groups of 128 programmable shader cores (CUDA cores) for a total of 8704 cores. It also features 272 tensor cores for AI and machine learning tasks and 68 RT cores [10] for accelerating real time ray tracing. These cores are paired with 272 texture units, 96 ROPs, 42 Polymorph engines, and 7 Raster engines, all of which are fixed function hardware. Clearly a large portion of the area on the GPU is being allocated to the 8704 programmable cores, the result of applications demanding more parallel compute power [10].

For GPUs, fast parallel computing is a result of the architecture's execution model. While not all GPU manufacturers use the same model, some form of Single Instruction Multiple Data (SIMD) or Single Instruction Multiple Thread (SIMT) is used. SIMD architectures, also commonly called vector processors, execute a single instruction on

multiple data in the same ALU. This model provides great parallel compute power when there is enough data to fill each instruction. If different operations need to happen on a per-data basis, SIMD ALUs lead to wasted area and power. Even worse, the programmer must handle this divergence in the code, leading to a more difficult programming experience. This has led some architectures to shift to a SIMT model. SIMT also allows a single instruction to be executed across different pieces of data, but it removes many of the programming difficulties of divergence by handling this in the hardware. The programmer simply writes a single program that will be executed on different pieces of data. The hardware spawns the right number of instances of this program (called threads) and schedules them in parallel across the cores of the GPU. If the threads diverge due to conditional statements, the GPU handles this in the hardware itself. Divergence still degrades performance and scheduling threads requires extra hardware, but the simplicity of the programming model provides a huge step over SIMD designs.

2.4 Vortex GPGPU

The work done in this thesis was performed in collaboration with the High Performance Computing group at the Georgia Institute of Technology (Georgia Tech). Their prior work, the Vortex GPGPU [18], was used as the foundation for this thesis. Vortex is an open-source GPGPU design created in Verilog and based on the RISC-V ISA. It implements a SIMT architecture and was designed to be instantiated on high-end Field Programmable Gate Arrays (FPGAs) for the purpose of architecture research. Vortex is a modular design, meaning that the user can specify a number of parameters about the architecture such as number of cores, threads, or caches before implementing the design on an FPGA. This allows the performance, area,

and power characteristics of different GPU configurations to be explored and their tradeoffs analyzed.

To implement a SIMT GPU design, Vortex uses a hierarchy of threads, warps and cores at the architecture level that enable a simple programming interface. The programmer can write a single scalar program as though they were operating on one piece of data and the hardware will handle execution of this scalar program across different pieces of data in parallel. Each instance of this scalar program is called a thread. Vortex organizes groups of independent threads into a warp (also called wavefronts). A warp is a group of threads that can be executed in parallel; they consist of the same program. Within the Vortex architecture, each programmable core supports a predetermined number of warps. Every clock cycle, the Vortex scheduler inside a core picks a valid warp to execute [12]. It interleaves different warps within the core to hide stalls from memory access. When a warp is chosen, the next instruction for the threads inside the warp is fetched and executed across all of the threads in parallel. Divergence of threads can be handled by the hardware using a thread mask. The thread mask allows the core to choose which threads inside the active warp should be executed when divergence between threads exists.

An example Vortex configuration might be four cores, four warps per core, and four threads per warp. Each clock cycle, all four cores independently choose one of their four warps to execute. That warp becomes the active warp and it executes the same instruction on all of its valid threads (via the thread mask) during the execute stage of the pipeline. Theoretically, such a configuration could execute sixteen instructions per cycle.

Each RISC-V core inside Vortex is a five-stage pipeline design (Figure 2.2). Supporting the concept of threads and warps for SIMT execution requires custom instruc-

tions to extend the RISC-V ISA. Vortex includes a total of six custom instructions:

- wspawn - activates a number of warps at a specific program's PC value
- tmc - activates or deactivates threads within a warp
- split - pushes relevant thread information to a stack for initializing divergence
- join - pops relevant thread information from a stack for reconvergence
- bar - synchronizes execution of different warps at a barrier location
- tex - specifies normalized coordinates for texture lookup

Vortex supports a modular design allowing the number of threads per warp, warps per core, cores per cluster, and clusters per processor to be specified by the user. L2 and L3 cache can also be enabled for larger designs.

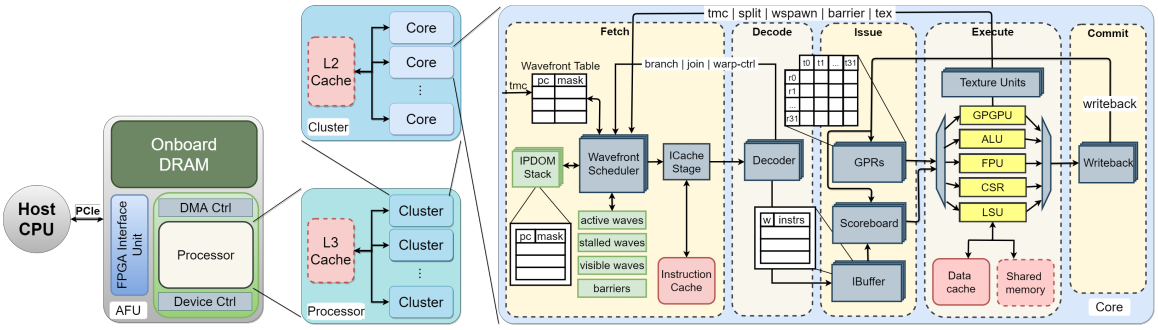


Figure 2.2: Overview of original Vortex GPGPU architecture with five stage pipeline, texture sampling, and scalable design [18].

The original design for Vortex supported the OpenGL-ES (embedded system) specification [4], a popular 3D rendering API. Supporting OpenGL-ES requires that Vortex can handle the 3D graphics rendering pipeline. The Geometry stage of the pipeline is handled by the host CPU with the results passed to Vortex to complete rasterization, fragment processing, and render output on the programmable cores. The original Vortex architecture included a dedicated texture unit to accelerate texture

filtering for both point and bilinear sampling. This involves loading and interpolating multiple values from nearby locations in memory. However, all other graphics functions such as rasterization and render output were implemented using software algorithms run on the programmable cores.

To utilize the Vortex hardware on a FPGA as a GPU accessible to the host CPU, an extensive software stack was created by the original design team. This stack centers around the use of the Intel Open Programmable Acceleration Engine (OPAE) C library to read and write data to the memory on the FPGA via a PCIe interconnect (Figure 2.3). Once the desired program and data is written to the FPGA's memory, a reset signal is sent to Vortex and processing begins. Once complete, the host retrieves the result from the FPGA's memory.

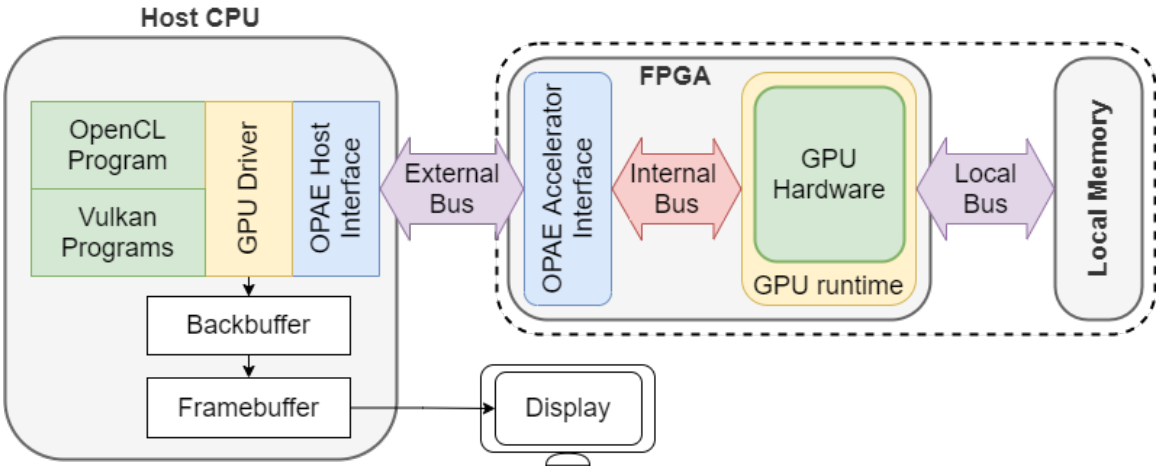


Figure 2.3: Vortex GPGPU driver interface using Intel OPAE to interface FPGA with host CPU across PCIe bus [18].

Vortex can also be simulated in the absence of an FPGA using one of three options. The first two options use a fast Verilog/SystemVerilog simulator called Verilator [14] to emulate Vortex with and without OPAE. These two options allow the user to choose whether or not they want to simulate Vortex as a separate PCIe device (VLSIM) or as a SoC device with a shared memory interface (RTLSIM) as seen in Figure 2.4. The

third option is a purpose-built, cycle accurate simulator (CAS) in C++ that avoids HDL simulation (SIMX).

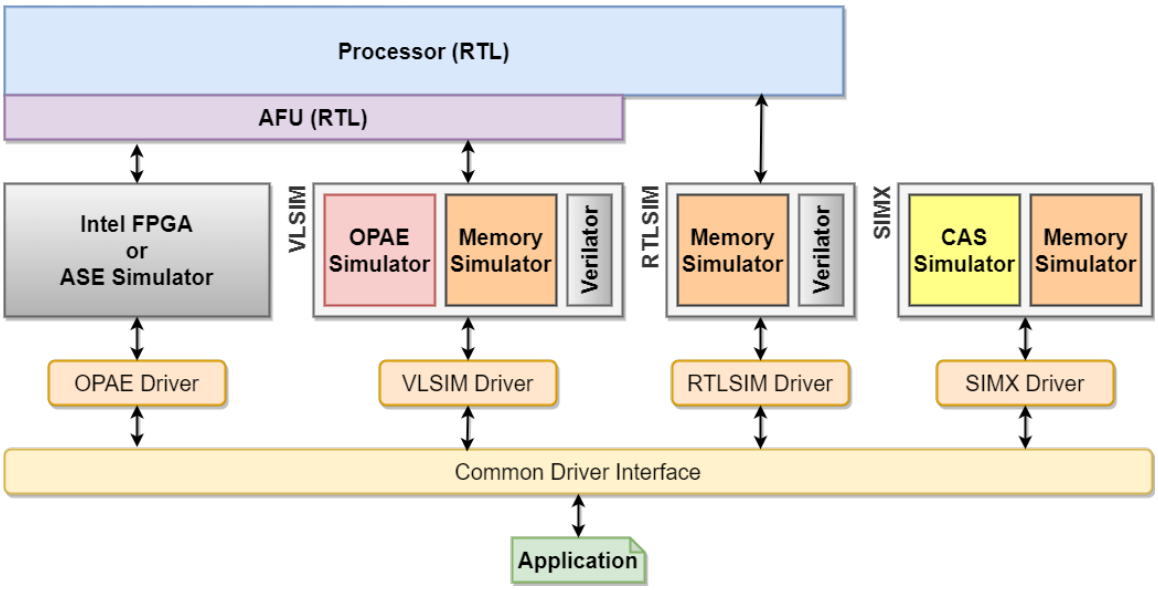


Figure 2.4: Vortex GPGPU simulation options, including FPGA implementation, Verilator with OPAE (VLSIM), Verilator without OPAE (RTLSIM), and the C++ simulator (SIMX) [18].

The original design for the Vortex GPGPU [18] contained very little specialized hardware for 3D graphics acceleration. This is a severe limitation on graphics performance since tasks such as rasterization, depth and stencil testing, and blending are frequently used, have little need for programmability, and can benefit greatly from hardware acceleration. These limitations motivated the design of fixed-function hardware units for the Vortex GPU to accelerate graphics rendering. Chapter three discusses these additions and their incorporation into the Vortex architecture.

2.5 Related Work

The following works explore topics related to extending the RISC-V ISA for GPU acceleration and implementing GPU architectures on FPGAs.

2.5.1 RISC-V Extension to Support GPGPU/GPU

Work done between the University of Science and Technology of China and Graphichina [22] demonstrates an extension to the RISC-V ISA for GPGPU tasks specifically with low power embedded systems in mind. The extension consists of a programmable vertex engine and a fixed-pipeline fragment engine to accelerate the graphics rendering pipeline. Allowing for programmable vertex shaders but limiting the fragment processing to fixed-function hardware strikes a balance between power consumption, area, and functionality. The programmable vertex engine uses SIMD instructions and special function units for transcendentals to accelerate vertex shaders. The design extends a single RISC-V core and was not scaled for a multi-core implementation unlike the new Vortex design presented in this thesis. The work in this paper does not implement a modular design and is unable to provide tools for exploring design tradeoffs within the GPU architecture.

2.5.2 FPGA Based GPUs

The FPGA-GPU developed by researchers at the University of Bath, UK [2] takes a different approach to implementing a graphics processor on an FPGA. Rather than a traditional multi-processor design using a SIMT or SIMD architecture, this FPGA-GPU is a pure logic design created to act as an accelerator. The traditionally programmable shader cores of a GPU are replaced with hard logic shaders that implement only a specific program. Although seemingly limiting, the idea is to utilize the reconfigurability of an FPGA to reprogram the hardware whenever a new shader is needed. The host CPU can manage resource utilization and dynamic reconfiguration of the FPGA SoC to maximize performance or efficiency. This design requires significantly more development time compared to a design like Vortex since the user must develop

hardware implementations of their shaders rather than the relative ease and flexibility of writing software shaders with a graphics API such as OpenGL. Furthermore, this work does not enable an exploration of GPU architecture tradeoffs for optimizing power, area, and performance.

Chapter 3

GRAPHICS ACCELERATION HARDWARE DESIGN

3.1 Project Goal

The Vortex GPGPU architecture completed in [18] supported graphics rendering for OpenGL-ES with hardware-based texture sampling and software algorithms for the rest of the graphics pipeline. The focus of this original effort centered around creating an open source SIMT GPGPU for parallel computing workloads, not graphics rendering tasks. Due to this focus, the original Vortex architecture was not optimized to render 3D graphics.

Following this original effort, a new project was undertaken by the team from Georgia Tech and Cal Poly to accelerate graphics rendering on Vortex with fixed function hardware. This effort explored the design tradeoffs between software and hardware graphics rendering and introduced new graphics extensions to the RISC-V ISA. To integrate these instructions with the fixed function hardware, a method for adding hardware extensions to the Vortex architecture was developed. The section below provides an overview of the hardware additions to Vortex followed by a detailed description of the author's specific hardware contributions to this effort. The final section uses a real world example to demonstrate how and where Vortex computes the stages of the graphics pipeline.

3.2 Overview of New Vortex Architecture

The original Vortex architecture divided the pipeline so the vertex processing for the geometry stage was done on the CPU and the rasterization and fragment stage were computed on Vortex (as software algorithms). A similar approach was chosen for the new Vortex architecture, but with a slight optimization. Since rasterization was to be accelerated with fixed function hardware, this portion of the pipeline would be on the Vortex hardware (similar to before). However, the triangle setup phase of rasterization (see Figure 2.1) would be kept on the CPU to reduce the number of repeated calculations. The logic for this decision is as follows: for the CPU to pass the primitives in an image to Vortex, it must compute expensive edge equations for each primitive to divide the primitives into manageable chunks called tiles. To avoid making Vortex recompute these edge equations (which are also used during triangle setup), both the process of tiling and triangle setup are done at the same time on the CPU. From there, the tiles are passed to Vortex for rasterization (now hardware accelerated) and the fragments created from this stage are given to the programmable cores on Vortex to compute the fragment shader. Finally, the last phases of the fragment stage are accelerated with fixed function hardware before the results are written back to the framebuffer. This new division of the graphics pipeline can be seen in Figure 3.1.

Supporting this new division of the graphics pipeline requires a few key additions to the Vortex architecture seen in Figure 2.2. The most obvious additions are a fixed function raster unit for rasterization and a fixed function render output unit (ROP) for the last phases of the fragment stage. Since these are fixed function extensions to the Vortex GPU, the programmable cores need a way to send requests and receive data from these units. To accomplish this task, a raster and ROP service block are

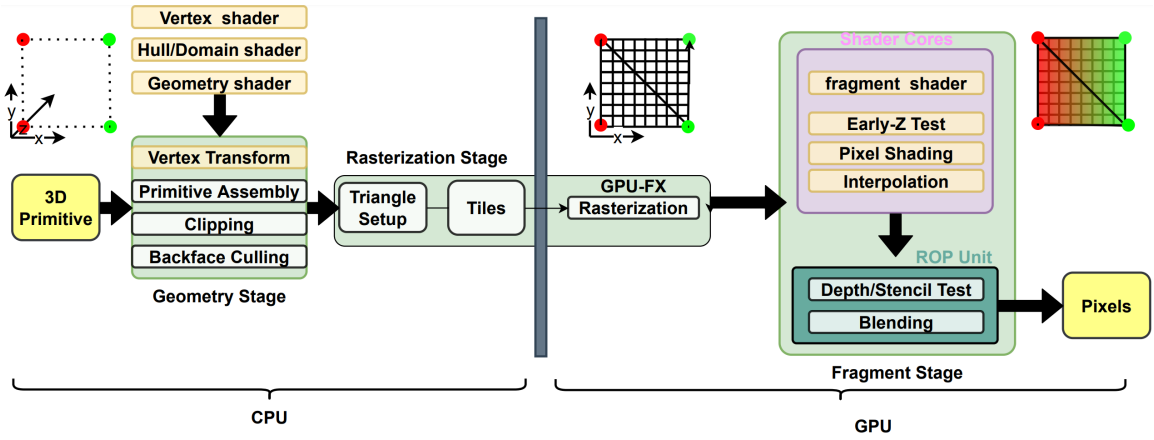


Figure 3.1: Vortex implementation of graphics pipeline divided into host CPU and GPU sections.

present inside each core near the texture unit. Two new instructions added to the RISC-V ISA activate these service blocks and direct them to send or receive the relevant data from the raster or ROP units that sit outside the cores. This new architecture can be seen in Figure 3.2. The following paragraphs further detail each hardware addition.

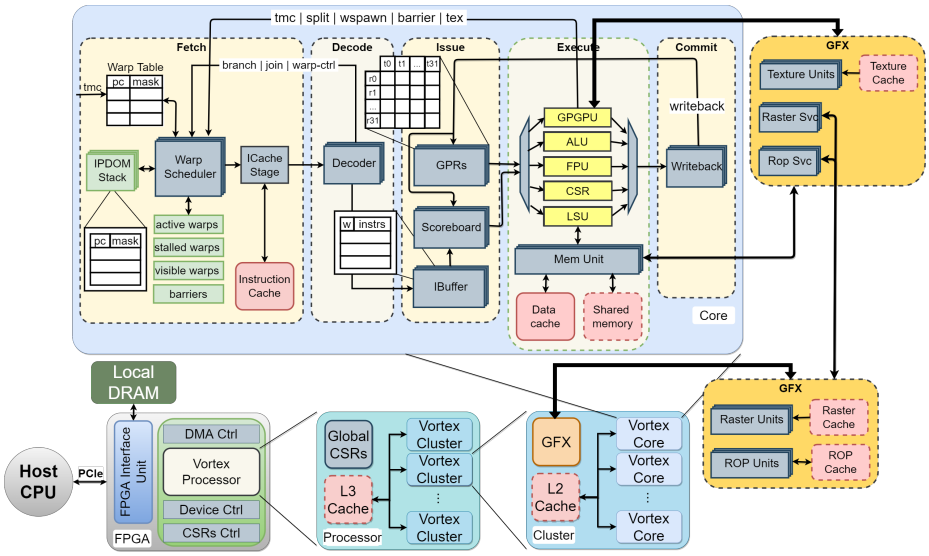


Figure 3.2: New Vortex GPU architecture with raster and ROP units added at the cluster level. Service blocks in each core direct requests to the raster and ROP units.

The raster unit in the new Vortex architecture is designed around a tile based rendering system. A tile based rendering system divides the frame window into sections called tiles after the geometry stage of the graphics pipeline. Each tile contains a subset of the primitives in the scene (determined by the edge equations mentioned earlier). The tiles are passed to the raster unit to be rasterized into fragments. The advantage of this method compared to a more traditional immediate mode rendering system is a reduction in bandwidth and memory required [1]: each tile is a subset of the scene and thus can be transferred and stored in smaller memories. The raster unit's tile processing hardware is held in a raster slice. Multiple raster slices can be instantiated to process tiles in parallel. Once a tile enters a raster slice, a recursive descent algorithm begins to break the tile into four equal sized chunks. Once these chunks are equal to the predetermined "block" size, these blocks are stored in a block queue until ready to enter the next stage of the raster pipe. The next stage contains parallel quad evaluator units, a quad being a two by two grid of fragments. A block is removed from the block queue and divided into the pieces of the correct size needed to generate a quad. These smaller pieces are passed to their own quad evaluators and processed in parallel. The quad evaluators determine whether a fragment is shaded by the current primitive and what the primitive's color contribution is to that fragment's color. The two by two fragment quad outputs are then placed in a quad queue to wait for a programmable core to fetch them. The design of the raster unit is displayed in Figure 3.3.

Supporting the raster unit is a configurable raster cache (RCACHE), a default two-way set associative cache with 4kB capacity and a 64 byte line size. The RCACHE is included to speed up the raster unit's read requests when fetching new tile data.

After the quads have been fetched by the programmable cores and operated on by the fragment shader, the resulting fragments are passed to the new ROP unit

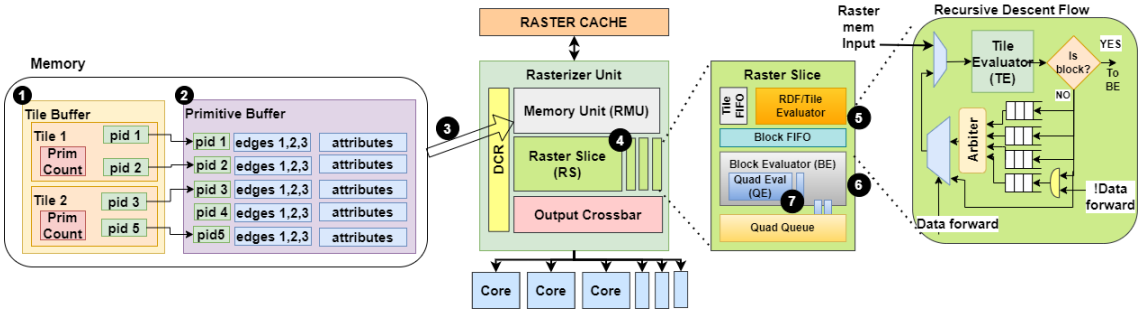


Figure 3.3: Architecture of Vortex raster unit with data structures, raster slices, and tile evaluators detailed.

for final tests and blending before being written to the framebuffer. The ROP unit can also be instantiated with multiple slices similar to the raster unit, allowing it to process multiple core requests at a time. Each core request is a collection of the requests from the core's individual threads. An ROP slice has as many lanes as there are threads in a core in order for a single slice to compute all of the thread requests in parallel. The primary task of the ROP unit is to perform the stencil test, depth test, and blending on each fragment when enabled by the current program. For the stencil test, the current fragment's stencil value (data similar to color) is compared to the stencil value in the current stencil buffer location. Any boolean comparison can be performed between these two values. If this test fails, the fragment is discarded. Similarly, for the depth test, the current fragment's depth value (another piece of data similar to color) is compared to the depth value at the current destination depth buffer location. If the test fails, the fragment is discarded. Operating in parallel to the stencil and depth tests is a multi-lane blending unit. The blending unit combines the current fragment's color, referred to as the source color, with the destination color stored in the framebuffer according to a blend equation or logical operation. The final color value from the blending unit and the results of the stencil and depth tests are then written back to the destination buffers in memory using the ROP's memory unit. The design of the ROP unit is displayed in Figure 3.4.

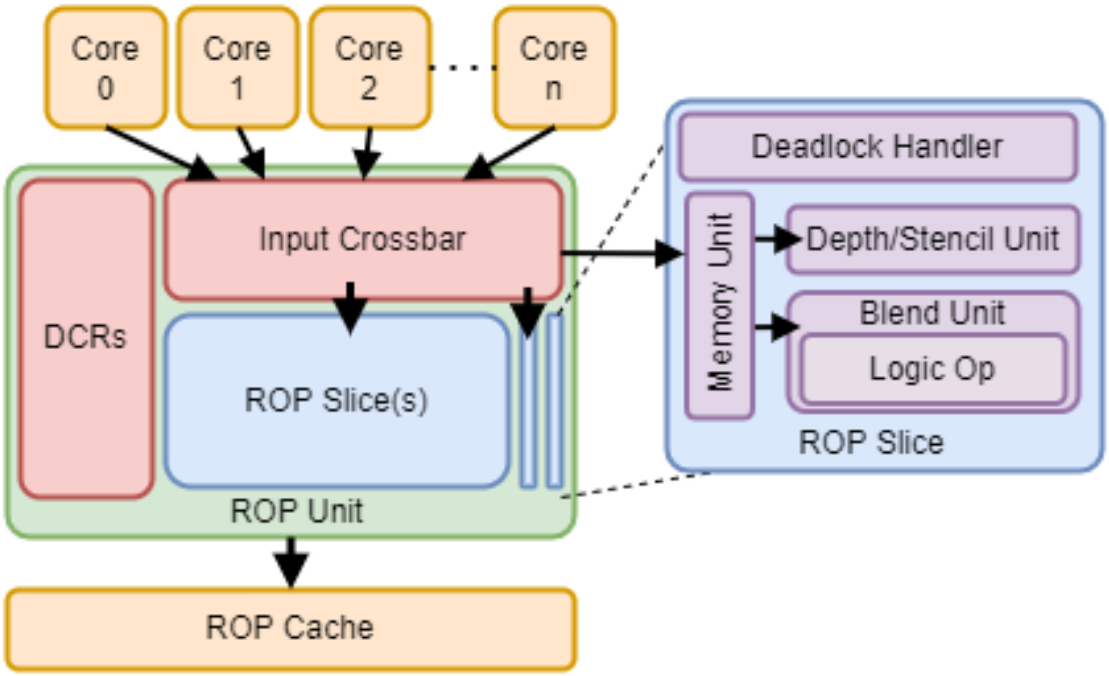


Figure 3.4: Architecture of Vortex ROP unit showing the parallel depth/stencil test and blend unit inside each slice.

Supporting the ROP unit is a configurable ROP cache (OCACHE), a default two-way set associative cache with 4kB capacity and a 64 byte line size. The OCACHE is included to speed up the ROP unit's read and write requests to the framebuffer.

Alongside the raster and ROP units come the server blocks that enable the programmable cores to send and receive requests from the Raster and ROP units. The raster and ROP service blocks lie inside the execute stage of the programmable core's pipeline. The raster service block retrieves a finished quad from the rasterizer and saves it into local registers to allow the core to begin executing the fragment shader. After the fragment shader is complete, the ROP service block sends the resulting fragments to the ROP unit's queue to be tested and blended before being written back to the framebuffer.

To allow the programmable cores to support these two hardware extensions, two new instructions were added to the RISC-V ISA:

- rast - raster load operation, activates raster service block in execute stage
- rop - ROP store operation, activities ROP service block in execute stage

A third instruction was added to accelerate interpolation of fragment attributes within a shader program:

- imadd - interpolation operation, performs integer multiply-add

The result of these instruction additions is a SIMT capable RISC-V core with hardware accelerated rasterization, stencil and depth testing, and blending.

3.3 Hardware Contributions

As noted, the new Vortex architecture incorporates a fixed function ROP unit to support testing and blending of fragments in the final stage of the graphics pipeline before writing the results to the framebuffer. Figure 3.4 displays the ROP unit with multiple slices connected to an input crossbar where each slice contains both a depth/stencil unit and a blending unit in parallel. The blending unit, input crossbar, and ROP cache additions are described further in this section.

3.3.1 ROP Blending Unit

A single lane of the blending unit combines the current fragment's color, referred to as the source color, with the destination color stored in the framebuffer according to a blend equation or logical operation. This process is shown in Figure 3.5. The blend equations use blend factors, determined by blend functions in stage ① of Figure 3.5, as weights based on the OpenGL specification [4]. If a logical operation is chosen

instead of a blend equation, a boolean operation will be performed directly with the source and destination color values.

Parallel blend equation hardware units compute separate color and alpha values using the blend factors calculated earlier as shown in Figure 3.5 stage ②. In the new Vortex architecture, the blending unit is pipelined and uses fixed-point math to compute the blend factors first, then blend equations or logical operations in a second stage. The actual blend equations are fixed-point implementations of multiply-add, multiply-subtract, reverse multiply-subtract, minimum, and maximum functions. These different modes take the pixel value from the fragment shader called the source value (R_s, G_s, B_s, A_s) and the current pixel value in the framebuffer called the destination value (R_d, G_d, B_d, A_d). The blend factors from the first stage also come in two sets: the source blend factors (S_r, S_g, S_b, S_a) and the destination blend factors (D_r, D_g, D_b, D_a). Equations 3.1-3.4 describe each blend mode.

$$R = f(R_s * S_r, R_d * D_r) \tag{3.1}$$

$$G = f(G_s * S_g, G_d * D_g) \tag{3.2}$$

$$B = f(B_s * S_b, B_d * D_b) \tag{3.3}$$

$$A = f(A_s * S_a, A_d * D_a) \tag{3.4}$$

$$where f = Add|Sub|RevSub|Min|Max$$

The alternative to these blend equations are the logical operations carried out by the LogicOp unit in stage ②. In this mode, each pixel's final color value is the result of a logical operation (and, or, xor, etc) between the source and destination pixel colors. Either a blend equation or a logical operation will be carried out to determine the final blended color in stage ③. This result from the blending unit and the results

of the depth-stencil test are then used to write back to the destination buffers in memory using the ROP's memory unit.

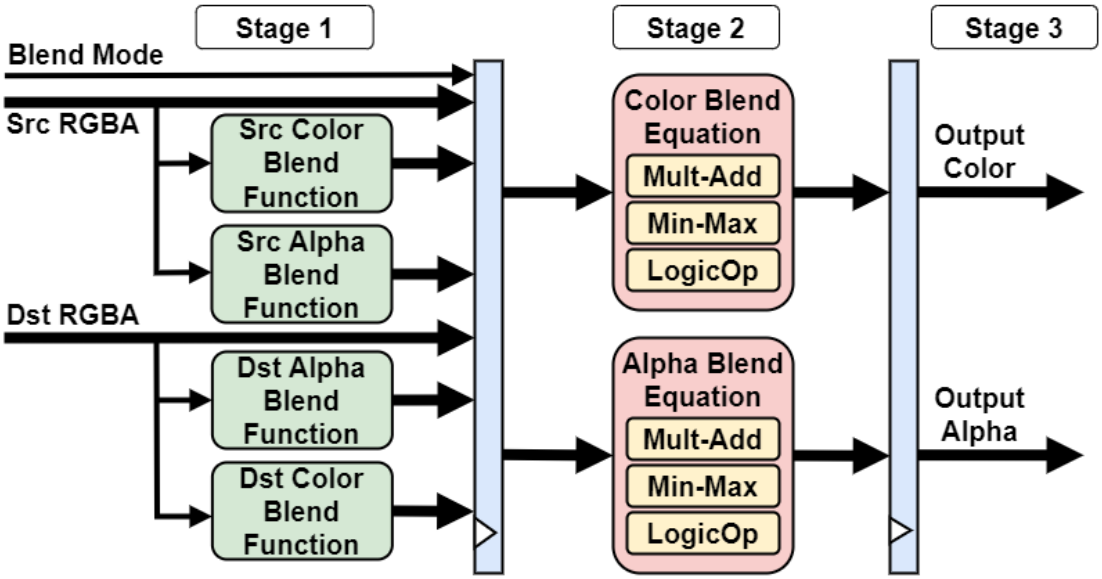


Figure 3.5: Design of blend unit inside ROP slice. Only one compute lane shown.

3.3.2 ROP Input Crossbar

As the ROP slices process fragments and write the results to memory, they are fed by the input crossbar seen in Figure 3.4. The input crossbar acts as an arbiter between the programmable cores and the ROP slices. As the cores send requests to the ROP unit, the crossbar chooses a core to serve and passes that core's request to a FIFO queue. When a raster slice is ready to process another set of fragments, it pops the next request off of the queue.

3.3.3 ROP Cache Additions

A private cache was added to the ROP unit with the intent of speeding up memory read and writes. Two additional features were added to the ROP cache system for modularity and performance monitoring.

The first addition was a cache bypass system. If the user specifies a Vortex configuration without an ROP cache, the ROP read and write requests are forwarded to main memory as Figure 3.6 demonstrates. This effort originally aimed to re-route ROP memory requests to L2 cache, but the data width of requests from the ROP unit and the programmable cores was not identical. Instead, a bypass parameter was added to the cache module in the Register Transfer Level (RTL) description. When bypass is set to one, the rest of the cache logic is optimized out leaving only a connection to main memory by which all incoming requests are passed.

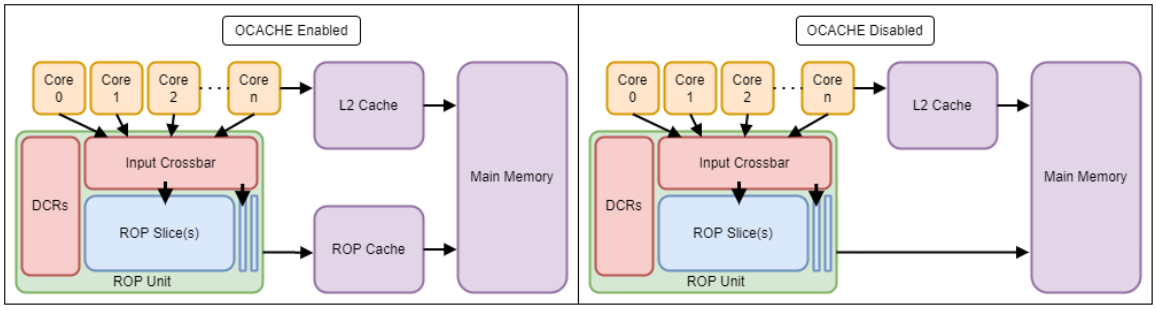


Figure 3.6: Overview of cache bypass design allowing ROP cache to be omitted in favor of directly accessing main memory.

The second addition to the ROP cache system was performance counters. These performance counters measured the total number of reads, writes, read misses, write misses, and total stall cycles. These values were stored in registers inside the ROP cache. To read these values and print performance metrics, the output of the registers was routed to the Configurable Status Registers (CSRs) inside the programmable cores. The CSRs are the primary interface used on the software side of Vortex to

read and write configuration values to the hardware. Once the performance metrics are routed to the CSRs, a software benchmark calls a CSR read function and grabs the values held in these registers.

3.4 Triangle Rendering Example

To demonstrate how the new fixed function hardware and programmable cores work together to render an image, a real world example of a simple triangle is presented here. Figure 3.1 provides a high-level overview of where each portion of the graphics pipeline is executed for the Vortex system. Following this process, example images and code using Vulkan [5] are presented to show a triangle rendered to the screen.

3.4.1 Geometry Stage

At the beginning of the rendering process, the simple triangle shown in Figure 3.8 is represented by three vertices with an x, y, and z coordinate. These vertices are held in memory on the host CPU side. The primary shader in the geometry stage of the graphics pipeline is the vertex shader. This program executes on the host CPU in the Vortex system. Figure 3.7 is an example of a vertex shader in Vulkan with a triangle explicitly declared [11]. The shader will run on a per vertex basis (in this case three times). Figure 3.8 shows the result of this shader running on the example triangle, transforming the vertices to screen space.

```
#version ---

layout(location = 0) out vec3 fragColor;

vec3 coordinates[3] = vec3[](
    vec3(0.0, -0.8, 0.0), // vertex #0
    vec3(-0.8, 0.8, 0.0), // vertex #1
    vec3(0.8, 0.8, 0.0)   // vertex #2
);

vec3 v_color[3] = vec3[](
    vec3(1.0, 1.0, 1.0), // black
    vec3(1.0, 0.0, 0.0), // red
    vec3(1.0, 1.0, 1.0)  // black
);

vec4 transform(vec3 coordinates, float w) {
    // rotate triangle coordinates
}

void main() {
    gl_Position = transform(coordinates[gl_VertexIndex], 1.0);
    fragColor = v_color[gl_VertexIndex];
}
```

Figure 3.7: Vertex shader code example. Host CPU executes this program on a each vertex.

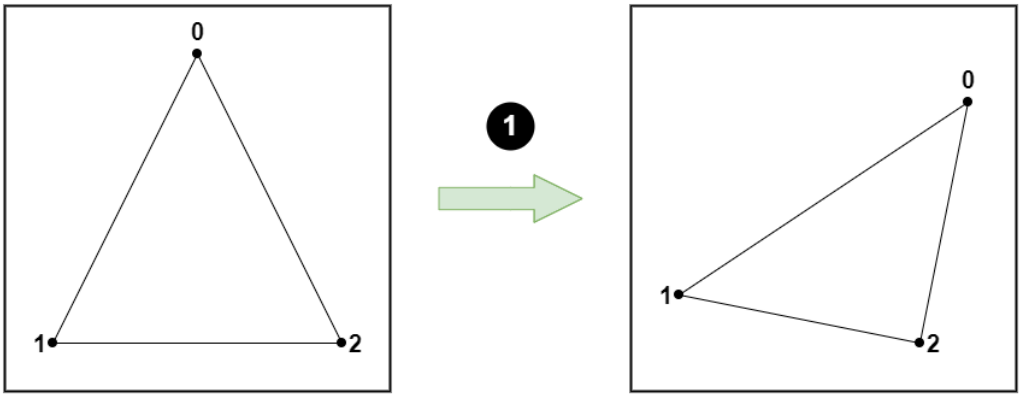


Figure 3.8: Visual example displaying results of geometry stage in graphics pipeline. Green arrow represents vertex shader.

3.4.2 Rasterization Stage

Once the vertex shader has finished, the rasterization stage begins. Still operating on the host CPU, the transformed triangle is broken up into tiles to implement tile based rendering. For this example, the triangle fits inside a single tile of size 8x8 that is copied to the Vortex GPU's memory. Once Vortex begins processing, it starts rasterizing this tile: the tile is broken down into equal quadrants until the quadrants are the size of a block. For this example, a block is 4x4 pixels. ① in Figure 3.9 displays one result of the recursive algorithm dividing the tile into block sized chunks. From here, ⑥ of Figure 3.3 breaks the block into 2x2 sized quads to be sent to the parallel quad evaluators ⑦. Each quad evaluator receives a quad similar to those seen after ② in Figure 3.9. These quad evaluators sample the quad, shading in any fragment whose center lies inside the triangle akin to ③ in Figure 3.9. The result for the entire tile can be seen after ④ in Figure 3.9. The individual quads, now a group of four fragments, are sent to the programmable cores for the fragment stage of the pipeline.

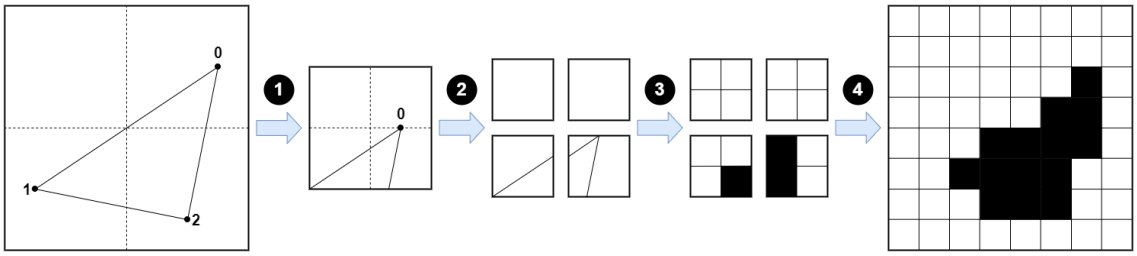


Figure 3.9: Visual example displaying results of rasterization stage in graphics pipeline. Each blue arrow represents a process within the raster unit hardware.

3.4.3 Fragment Stage

The first part of the fragment stage, the fragment shader, executes on the programmable cores inside of Vortex. The fragment shader is a program that colors the fragments in the image. Figure 3.10 is an example of a simple fragment shader in

Vulkan [11]. The color of each vertex is specified in the vertex shader and passed to the fragment shader where the color of each fragment is calculated as an interpolation of the vertex colors. ① in Figure 3.11 demonstrates the result of this simple fragment shader, where the color of the right most vertices was set to black, and the color of the left vertex was set to red. The result is a gradient from red to black.

```
#version ---

layout(location = 0) in vec3 fragColor;

layout(location = 0) out vec4 outColor;

void main() {
    outColor = vec4(fragColor, 1.0);
}
```

Figure 3.10: Fragment shader code example. Each thread in Vortex's programmable cores executes this program on a different fragment.

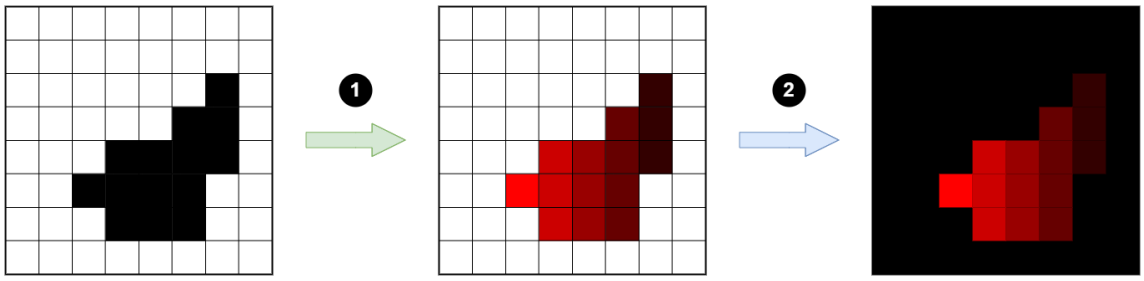


Figure 3.11: Visual example displaying results of fragment stage in graphics pipeline. The green arrow represents the fragment shader, the blue arrow represents a process within the ROP unit hardware.

Finally, the shaded fragments are passed to the ROP unit, where the final testing and blending take place. In this example, the stencil test and depth test are not needed. However, if another triangle was in this scene, the GPU would need to know which is in front and how to blend their colors if they overlap. For blending, the default background color (black) is replaced by rather than being combined with the new fragment color. The final result is then written to the framebuffer.

Chapter 4

RESULTS AND ANALYSIS

The Vortex system offers four options for implementing and testing the performance of different hardware configurations. Figure 2.4 shows these options in more detail. The results gathered for this thesis use the RTLSIM option with Verilator (a Verilog Simulator) to implement the Vortex GPU. This option was chosen since Vortex requires a large FPGA with a PCIe bus to implement in real hardware. Devices with these specifications were too expensive to acquire for these tests, but using these types of FPGAs remotely through industry university programs is a future goal.

For measuring graphics rendering performance on the Vortex GPU, eight different benchmarks were used from the PowerVR SDK [6], each consisting of a different "trace". A trace simply refers to a specific collection of primitives and the drawing commands needed to form the resulting image. The eight benchmarks are Vase, FiltmTV, Skybox, Coverflow, Evilskull, Polybump, Tekkaman, and Carnival. A sample render for each benchmark is shown in Figure 4.1. Each benchmark consists of a number of draw calls (calls to the API to draw the primitives to the screen), vertices, primitives, textures, and whether depth/stencil testing or blending is enabled. Table 4.1 displays this data for each of the eight benchmarks.

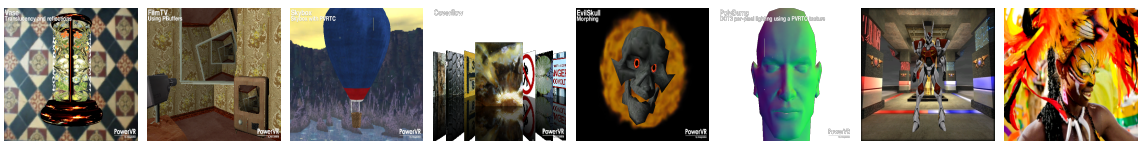


Figure 4.1: Sample renders of each benchmark (vase, filmtv, skybox, coverflow, evilskull, polybump, tekkaman, carnival) [6]

For each benchmark, the performance of the Vortex GPU was the desired metric. This allowed for the Geometry Stage of the graphics pipeline to be pre-computed for each trace since this portion executes on the host CPU. Therefore, the input data for each benchmark was a file containing these pre-computed vertices grouped into primitives ready to be broken into tiles. These tiles were then copied to Vortex's memory for the GPU to begin the benchmark.

Besides the hardware configuration for Vortex, a number of parameters exist for the benchmarks themselves. The only ones adjusted for these experiments were the width and height of the output rendered image (specified in number of pixels).

Table 4.1: Benchmark descriptions in terms of draw calls, number of vertices, number of primitives, number of textures, depth/stencil test enable, and blending enable

Name	Draws	Verts	Prims	Tex	D/S	Blend
vase	6	1198	1643	5	1	1
filmtv	16	1282	1086	11	1	1
skybox	4	704	791	9	1	1
coverflow	11	157	73	13	1	1
evilskull	7	566	699	5	1	1
polybump	3	332	276	3	1	1
tekkaman	2	2269	757	2	1	0
carnival	1	4	2	1	1	0

4.1 Results

The Vortex GPU architecture was designed to be modular, allowing the user to specify different hardware configurations. The following sections describe the different hardware configurations tested and the results gathered from the graphics rendering benchmarks.

4.1.1 Varying Number of Cores, Warps and Threads

The first experiment run on the Vortex GPU was a look into how the total number of programmable cores affects graphics rendering performance. The number of cores was set to 1, 2, 4, and 8 while the thread and warp count were kept at a constant 4. The size of the output image was set to 64x64 pixels. Each of the eight graphics benchmarks was then run on these configurations and the Instructions Per Clock (IPC) was gathered from each. The results are shown in Figure 4.2.

The next experiment followed the same pattern, but this time the number of warps varied between 2, 4, and 8 while the core and thread count were kept constant at 4. Attempting to synthesize Vortex with only one warp fails to compile since the Vortex architecture relies on having at least two warps per core to interleave execution of threads each clock cycle. The size of the output image was again set to 64x64 pixels. Each of the eight graphics benchmarks was then run on these configurations and the IPC was gathered from each. The results are shown in Figure 4.3.

Finally, the number of threads is varied between 2, 4, and 8 while core and warp count are kept at a constant 4. Similar to varying warps, Vortex cannot synthesize a configuration where each warp only has one thread. The size of the output image was set to 64x64 pixels; the IPC results are shown in Figure 4.4.

4.1.2 Software vs Hardware Rendering

Another set of experiments were run to compare the performance of the Vortex GPU with and without hardware accelerated graphics rendering. These experiments tested the performance of the Vortex GPGPU before a fixed function rasterizer, ROP, and interpolation instruction were added and compared the results to the Vortex GPU

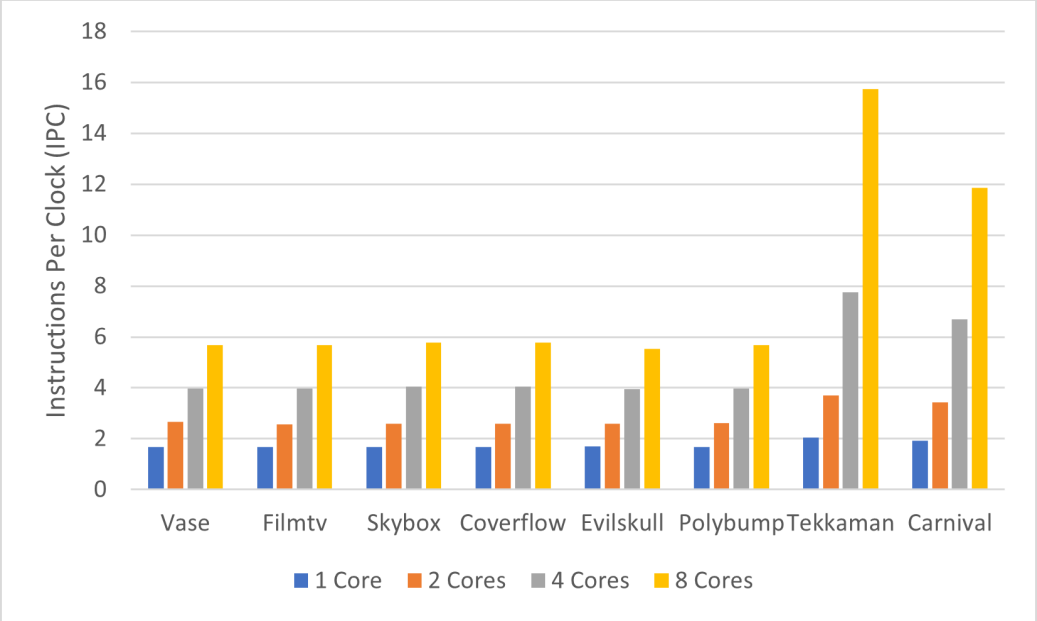


Figure 4.2: Instructions Per Clock (IPC) vs Number of Cores for each of the eight benchmarks. As the number of cores is doubled, IPC scales by an average factor of 1.6 times.

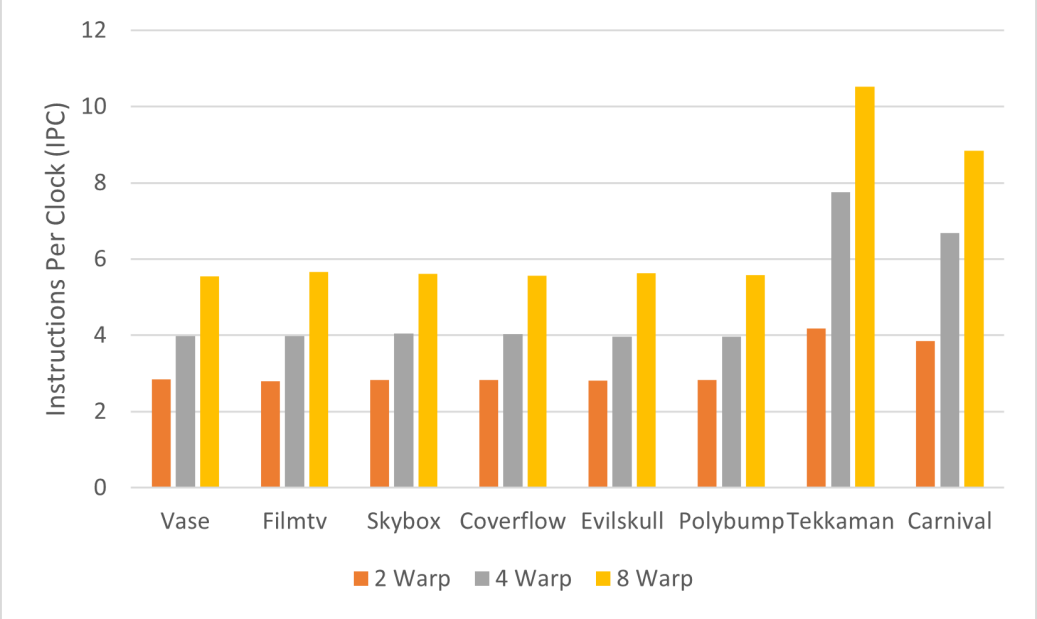


Figure 4.3: Instructions Per Clock (IPC) vs Number of Warps Per Core for each of the eight benchmarks. As the number of warps is doubled, IPC scales by an average factor of 1.45 times. Verilator failed to compile GPU when number of warps was set to one.

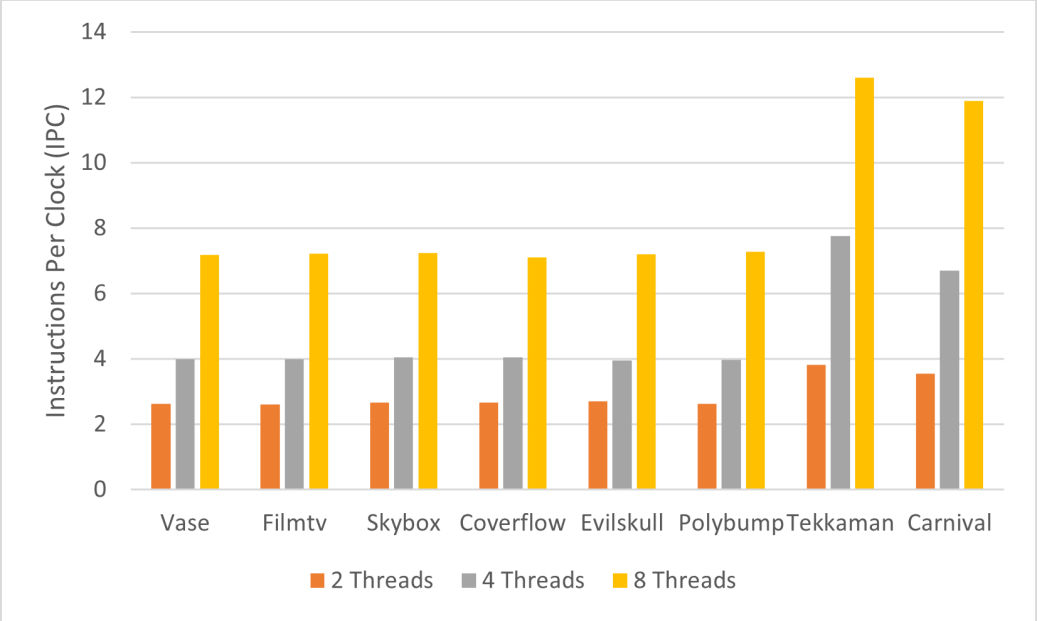


Figure 4.4: Instructions Per Clock (IPC) vs Number of Threads Per Warp for each of the eight benchmarks. As the number of threads is doubled, IPC scales by an average factor of 1.7 times. Verilator failed to compile GPU when number of threads was set to one.

with these hardware additions. To capture these results, each of the eight benchmarks was run with a variable number of cores (1, 2, 4, and 8) in two different modes. The first mode removed the hardware rasterizer, ROP, and interpolation instruction. These functions were instead implemented as software algorithms and executed on the programmable cores. The second mode fully enabled the hardware rasterizer, ROP, and interpolation instruction to represent the final design for the Vortex GPU with fixed function hardware. The size of the output image was set to 64x64 pixels. The number of warps and threads was held at a constant value of four. Rather than measuring performance by the IPC metric, the total number of cycles required to complete the benchmark was measured. This decision was made for a couple of reasons. First, the software algorithm approach for rendering will use a completely different program than the hardware accelerated approach resulting in an unequal number of instructions. Furthermore, the hardware accelerated approach does much of its work inside the fixed function raster and ROP units where instructions do not

exist and are thus not counted. This makes the total number of cycles (essentially a measure of runtime) a better metric for comparison between the two approaches.

Figures 4.5, 4.6, and 4.7 display the results for the Filmtv, Tekkaman, and Carnival benchmarks. The other five benchmark results can be found in Appendix A.

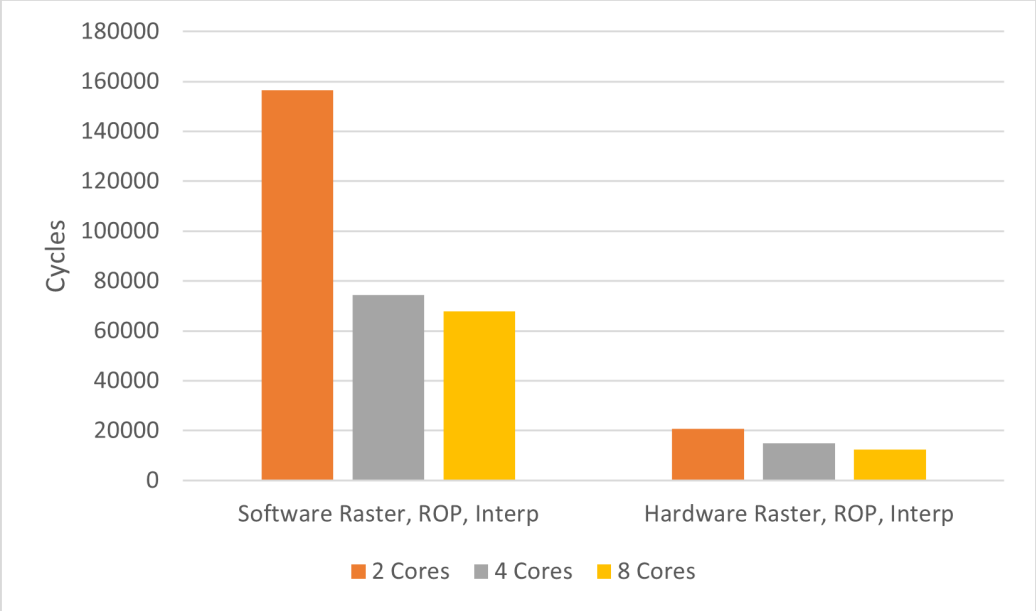


Figure 4.5: Total number of clock cycles to complete FilmTV benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.5 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.

4.1.3 Adding a Raster and ROP Cache

An important consideration for the Vortex architecture in terms of both area and performance is the cache sub-system. The modularity of Vortex allows the user to specify what caches should be enabled or disabled. The raster unit cache and ROP unit cache were of particular interest given the inherent potential to improve graphics rendering performance. Therefore, each of the eight benchmarks was run on the Vortex GPU with the following cache configurations: both raster and ROP cache

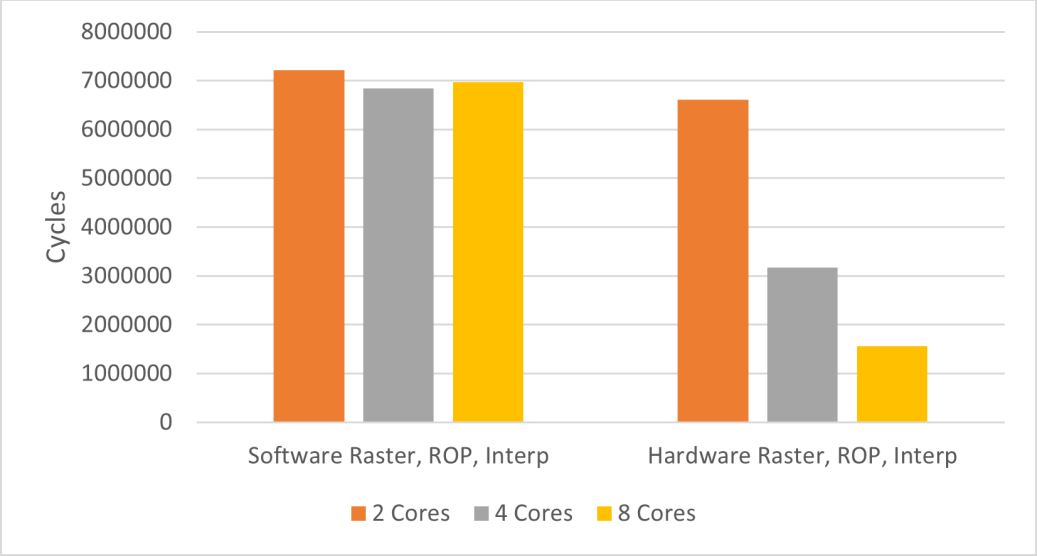


Figure 4.6: Total number of clock cycles to complete Tekkaman benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 1.1 to 4.5 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.

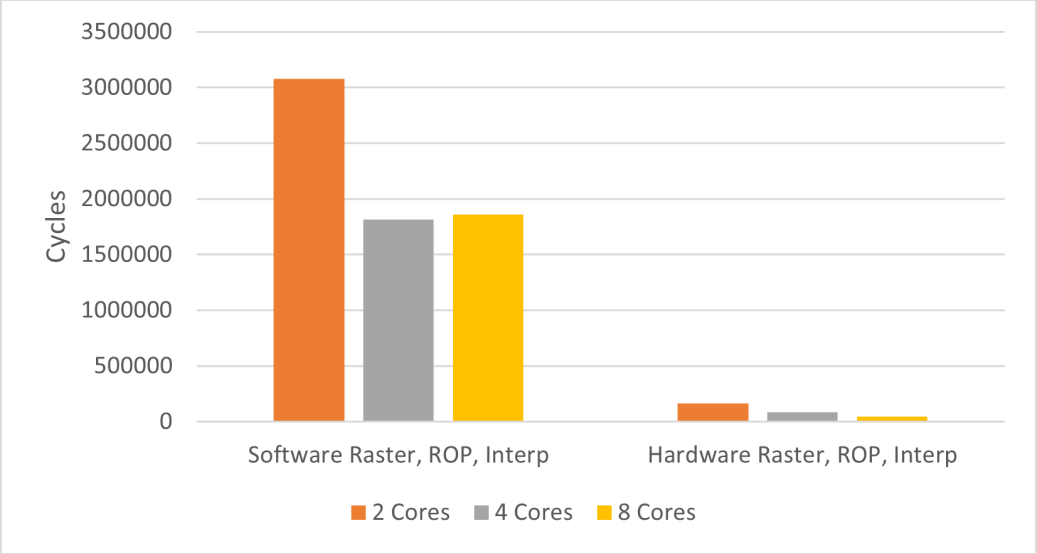


Figure 4.7: Total number of clock cycles to complete Carnival benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 19 to 38 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.

enabled, only raster cache enabled, only ROP cache enabled, and neither raster nor ROP cache enabled. The output image size was set to 64x64 and the number of cores, warps, and threads was set to four. The performance metric measured was IPC. Figure 4.8 shows the results of this experiment.

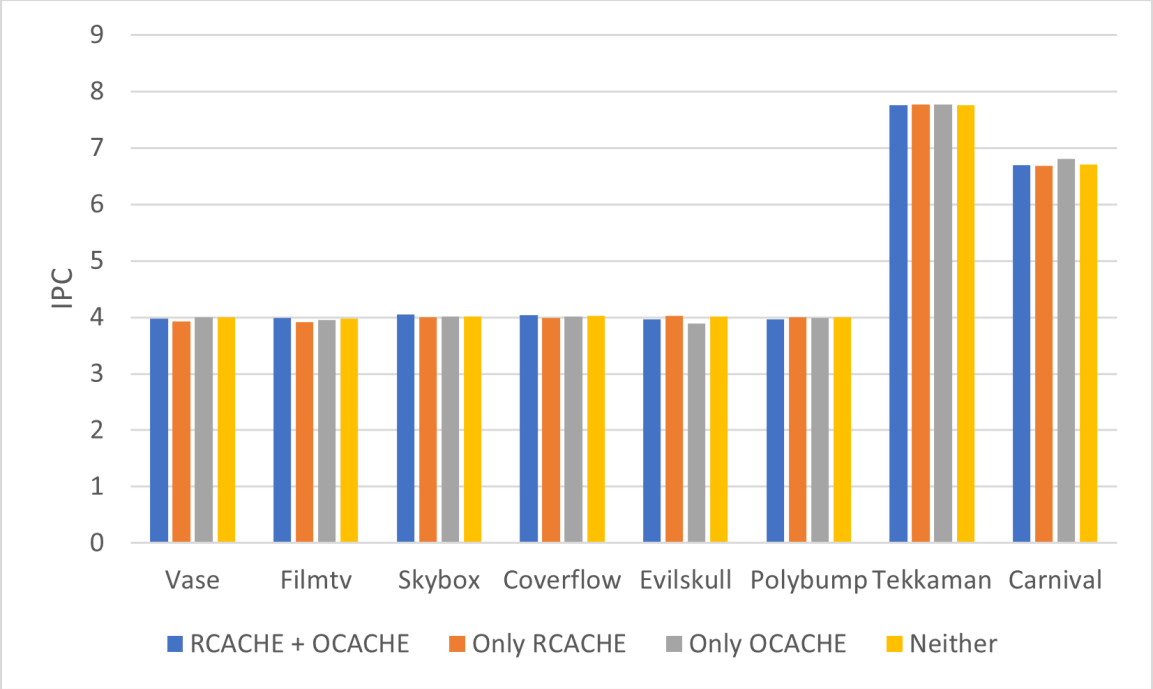


Figure 4.8: Instructions Per Clock (IPC) vs Type of Cache Configuration for each of the eight benchmarks. All results within the same benchmark stayed within 3% of each other.

Another experiment was also conducted to test the performance benefits of the raster and ROP cache. This time, only the Carnival benchmark was used while the other parameters were kept the same as as the previous experiment. The independent variable was the size of each tile in pixels being fetched from the raster unit cache and fed to the rasterizer. The larger the tile size, the more data the raster unit must fetch from memory at a given time, potentially allowing the cache to provide a meaningful speedup. The resulting IPC from this benchmark was then measured. These results can be seen in Figure 4.9.

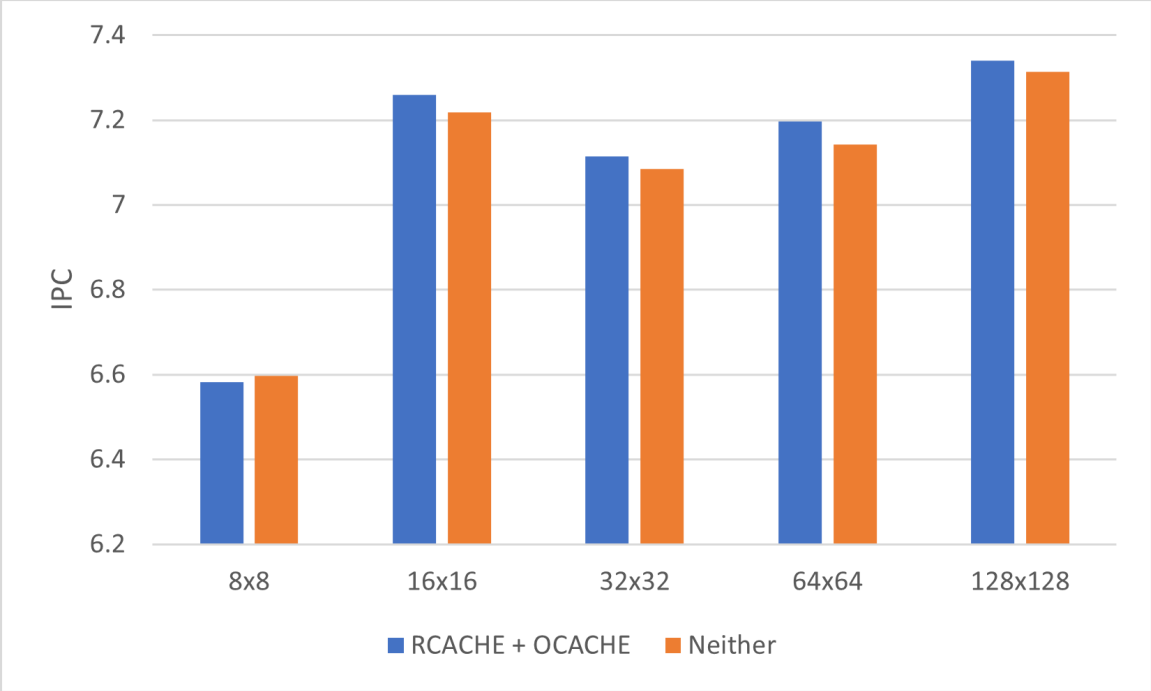


Figure 4.9: Instructions Per Clock (IPC) vs Type of Cache Configuration for various raster tile sizes using the Carnival benchmark. Above an 8x8 tile size, RCACHE + OCACHE configuration consistently performs faster, but by less than 1% across the board.

4.1.4 Varying number of Raster/ROP Slices

As mentioned in the previous chapter, the Vortex microarchitecture also allows the raster and ROP units to scale in the form of slices. By default, a raster or ROP unit consists of one slice capable of computing one tile's worth of fragments (for the raster unit) or one core's worth of depth/stencil and blending requests (for the ROP unit). Figure 4.10 shows the performance of the Vortex GPU with one, two, or four raster and ROP slices as the number of programmable cores was increased from one to eight using the Vase benchmark. The number of warps and threads was kept at a constant four and the output image size was set to 64x64. The performance of each configuration was measured in IPC.

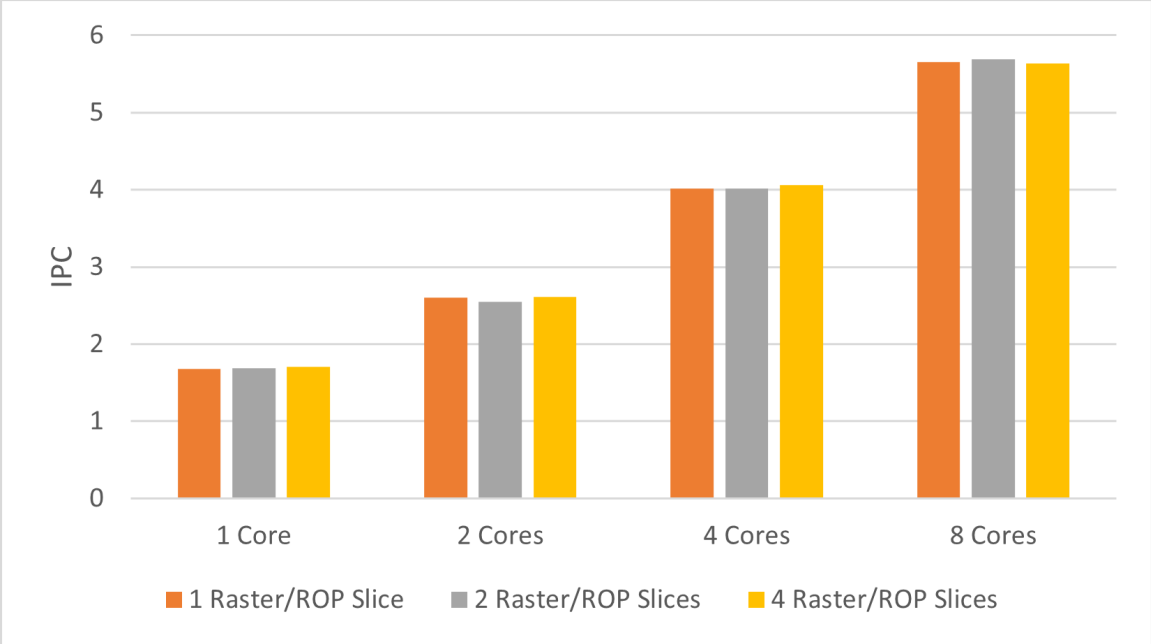


Figure 4.10: Instructions Per Clock (IPC) vs Number of Raster/ROP Slices for various core configurations using the Vase benchmark. No significant difference observed between different numbers of slices.

4.1.5 Varying Raster Tile Size

Since the Vortex GPU uses a tile based rendering architecture, another parameter available to modify is the tile size and block size of the raster unit. The default value for the tile size is 16x16 while the default value for the block size is 4x4. Figure 4.11 displays the results for the Carnival benchmark as the tile size is increased from 8x8 to 128x128. For each configuration, the block size was always set to a factor of four below the tile size. For example, if the tile size was 64x64, the block size was set to 16x16. The number of cores, warps, and threads was set to four. The output image size was set to 128x128 this time in order to test larger tile sizes. The performance for each configuration was measured in IPC.

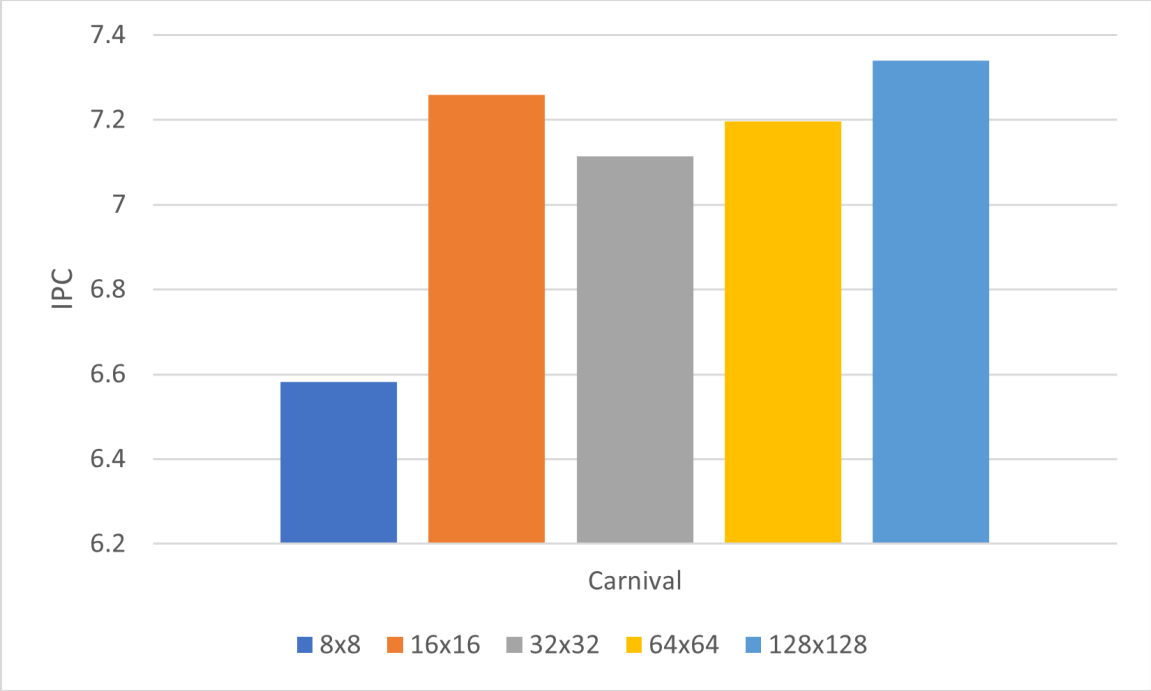


Figure 4.11: Instructions Per Clock (IPC) vs Raster Tile Size using the Carnival benchmark. Tile size of 128x128 perform best while a tile size of 8x8 performs nearly 10% slower.

4.1.6 Area usage

The area usage of the different components within the Vortex GPU is another useful metric to put the performance results in perspective. To obtain area cost of different components, Vortex was implemented on an FPGA and the resulting Look up Table (LUT), logic register, and Digital Signal Processing (DSP) slice utilization was obtained. A single core, four warp, and four thread configuration was used with a single slice for the raster and ROP units. The tile size and block size were set to 16x16 and 4x4 respectively and the raster cache (RCACHE) and ROP cache (OCACHE) were both enabled. Figure 4.12 shows the percent area usage of each component in terms of LUTs, Figure 4.13 shows the percent area usage in terms of logic registers, and Figure 4.14 shows the percent area usage in terms of DSP slices.

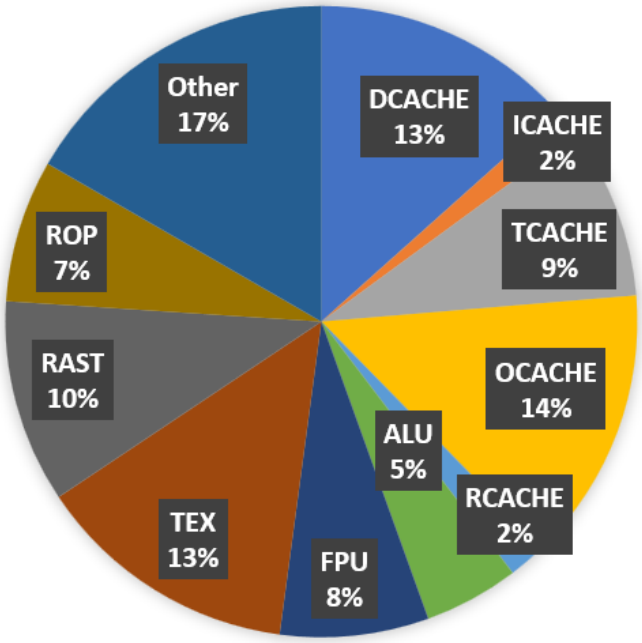


Figure 4.12: Area cost of Vortex GPU measured in Combinational ALUTs utilized per unit. A single core, four warp, four thread configuration was used.

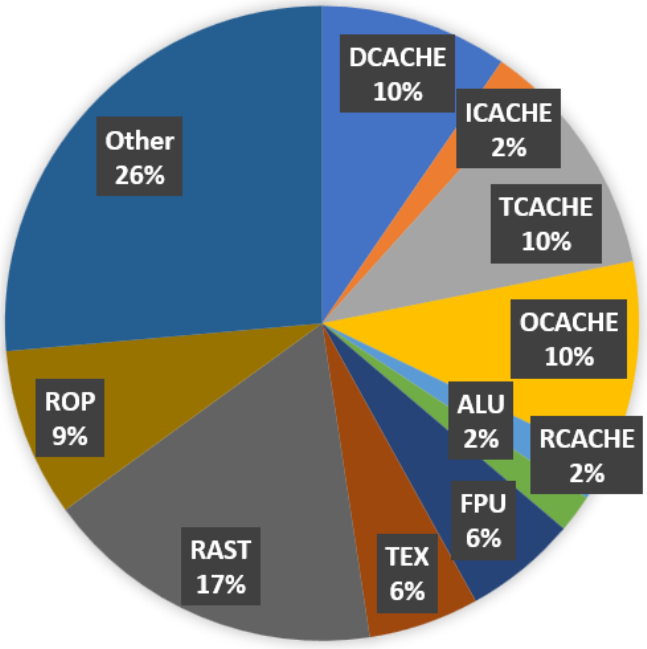


Figure 4.13: Area cost of Vortex GPU measured in Logic Registers utilized per unit. A single core, four warp, four thread configuration was used.

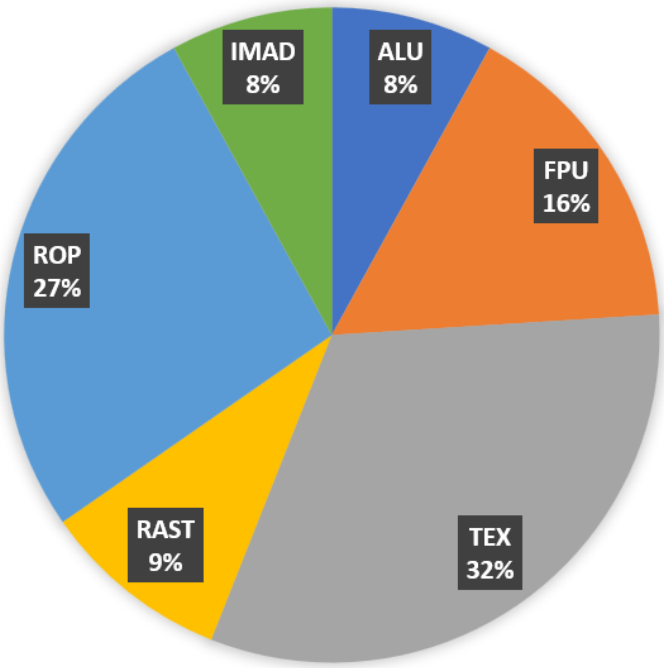


Figure 4.14: Area cost of Vortex GPU measured in DSP Slices utilized per unit. A single core, four warp, four thread configuration was used.

4.2 Analysis

4.2.1 Varying Number of Cores, Warps and Threads

The experiments for measuring the performance of different core, warp, and thread configurations provides the data for what is essentially an optimization problem. These three architectural parameters are the primary methods for scaling the Vortex GPU up in performance. However, this consumes area on the FPGA so it is ideal to know which of the three parameters provides the best performance improvement for a given increase in area.

Before this analysis, however, a couple outliers need to be addressed. The IPC results for the Tekkaman and Carnival benchmarks were consistently higher across all experiments run. Looking at Table 4.1, these two benchmarks are also the only ones that do not use blending. Blending happens in the ROP unit and it requires a series of multiply-add operations that contribute to additional runtime. Without blending, the Tekkaman and Carnival benchmarks are able to process more fragments without the ROP queue filling up with requests.

Even with these two outliers, a general trend in performance holds across all three experiments. First, the results in Figure 4.2 demonstrate an average of 1.6 times speedup as the number of cores is doubled. It should be noted that going from four cores to eight cores showed the smallest average speedup, closer to 1.54 times. Adding more cores will gradually produce a smaller and smaller return on investment.

Next, the results in Figure 4.3 display an average 1.45 times speedup as the number of warps is doubled. The same trend is noted as the number of warps moves from four to eight and the average speedup slides to 1.39 times. An increase in warps above eight will likely see a continued reduction in added performance.

Finally, the results in Figure 4.4 show an average 1.7 times speedup as the number of threads is doubled. It is important to note that the trend observed for increasing cores and warps is not seen with increasing threads; moving from four to eight threads resulted in the largest performance gain of a 1.78 times speedup.

Based on performance alone, adding more threads will, on average, result in a larger speedup than adding more cores or warps. These results make sense given that increasing the number of threads is similar to increasing the SIMD width of the core. If divergence among threads is low, increasing the effective SIMD width will increase performance proportionally. This condition is often met for graphics rendering tasks where the same program is executed on the many fragments of an image.

4.2.2 Software vs Hardware Rendering

Comparing the performance of Vortex before and after the addition of graphics hardware acceleration shows no surprises. Figure 4.5 demonstrates a common pattern seen in seven of the eight benchmarks. Having fixed function hardware for the raster unit and ROP unit along with a dedicated interpolation instruction produces a best case speedup of nearly eight times when using two cores and a worst case speedup of five times when using four cores. This pattern is only broken by the Tekkaman benchmark in Figure 4.6 where a higher core of four and eight is needed to see the benefits of the hardware acceleration.

These results confirm the well known fact that dedicated hardware implemented efficiently will often outperform the same algorithm running in software. The overhead added from executing instructions in the core's pipeline cannot compete with a fixed-function pipeline.

4.2.3 Adding a Raster and ROP Cache

While it might seem that a private cache for the raster and ROP units would benefit performance in graphics rendering, Figure 4.8 demonstrates that this assumption does not hold for any of the benchmarks. The largest performance difference is found in the Evilskull benchmark, where having only an ROP cache actually performs 3.5% worse than having neither cache.

These results make sense upon closer inspection. Unlike the programmable cores, the raster and ROP units do not take advantage of temporal locality. Each of these units takes in data from memory, runs it through its fixed-function pipeline, and writes the new data back to memory. The ROP unit does read from the current value in the framebuffer everytime blending occurs, and if it were to process multiple fragments with the same pixel location back to back, a performance benefit might be noticed since it would be reading the same value repeatedly. However, with the number of fragments being generated and sent to the ROP, the odds of this scenario occurring are unlikely. The value in the framebuffer for a particular pixel location will likely be evicted before it is needed again.

For the raster unit, accesses to memory are focused on the current tile being processed. Once that tile is done, the raster unit no longer needs that data. If, however, the size of the tile is increased so that the raster unit is accessing a larger chunk of data during a given interval, the case for having the raster cache could be made. Smaller tile sizes might result in the cache spending most of its time retrieving the next tile in memory but a larger tile size would allow the cache to fetch bigger chunks and thus result in a lower miss ratio. Figure 4.9 shows the results of an experiment used to test this possibility. Increasing the tile size above 8x8 pixels does

show a performance improvement when the raster and ROP caches are present. Even so, the difference is less than 1%.

4.2.4 Varying number of Raster/ROP Slices

The results in Figure 4.10 make it clear that additional raster and ROP slices do not benefit the system's rendering performance. Increasing the number of slices seemed promising theoretically due to the nature of the architecture. A raster slice computes a tile's worth of fragments and services these results to all the cores in its cluster. Similarly, an ROP slice services all of the requests for depth/stencil testing and blending from every core in its cluster as well. If the number of cores in a cluster increases, a single slice could become a bottleneck in either situation. However, given these results, it would seem eight cores is not enough to tax a single raster or ROP slice.

4.2.5 Varying Raster Tile Size

Changing the raster tile and block size is an optimization problem between area and performance. Inside the rasterizer, each tile is recursively broken down into four equal parts until the resulting parts equal the size of a block. To process a block, the architecture contains quad evaluators that compute the equations for 2x2 grids of pixels. The number of quad evaluators equals the total block area divided by two (a block size of 16x16 requires 64 quad evaluators). This means the raster unit will spend most of its time recursively breaking apart tiles into blocks if the block size is too small compared to the tile size. On the other hand, if the block size is made too large, the number of quad evaluators in the rasterizer must increase, consuming more area. For the results shown in Figure 4.11, the block size was always kept at a factor of four below the tile size to obtain performance results from the simulations in

a reasonable amount of time. Therefore, this experiment aimed to observe if a larger tile size produced a significant speedup to justify the increase in area of the raster unit as a whole.

From these results, the optimal tile size is clearly 16x16 (with a resulting block size of 4x4). This size provides a 10% speedup over the 8x8 tile size while only being outperformed by the much larger 128x128 configuration by 1.1%. The 16x16 configuration becomes even more appealing when the relative area cost is considered. In a 16x16 tile size configuration, the raster unit requires 4 quad evaluators. When implemented on an FPGA at this size, each quad evaluator makes up approximately 5% of the total Look up Tables (LUTs) and about 10% of the total registers used by the raster unit. The number of quad evaluators needed for a 128x128 configuration is 256. Such a large increase in area cannot justify the 1.1% speedup, but an increase from 1 to 4 quad evaluators (going from 8x8 to 16x16 tile size) is reasonable for a 10% speedup.

4.2.6 Area usage

Accelerating graphics rendering with fixed function hardware in the Vortex architecture comes at the cost of area usage. From Figure 4.12, the raster and ROP unit additions make up 17% of the total LUT utilization, 33% if their respective caches are included. In terms of logic registers, Figure 4.13 shows the raster and ROP units make up 26% of the utilization and 38% with the two caches added in. The DSP slice utilization tells a similar story, with the raster and ROP units using 36% of the total slices as per Figure 4.14.

Overall, the raster and ROP units make up a little over one third the total area in these three categories, but from the results in Figure 4.5, the speedup from software

to hardware rendering can range anywhere from 5 to 7.5 times. From Figure 4.8, the addition of an RCACHE and OCACHE was found to add no real performance improvement. Removing these makes the added raster and ROP units even more enticing as their area in terms of LUTs drops to about 20% utilization and to 30% utilization in terms of registers for the same 5 to 7.5 times speedup.

Chapter 5

CONCLUSION

It is little surprise that the addition of fixed function hardware to accelerate graphics rendering outperforms a purely software approach, a conclusion seen in Figure 4.5. More importantly, the new architecture for Vortex, with support for popular graphics specifications such as OpenGL and Vulkan, provides a framework for implementing and testing many different GPU configurations using real world graphics rendering scenarios. This feature was explored in the results and analysis section, where a varying number of cores, warps, threads, caches, raster/ROP slices, and tiles were implemented for the sake of determining an optimal design in terms of area and performance. From these experiments, a number of interesting architectural conclusions can be drawn. Firstly, when choosing to scale up the Vortex architecture for more performance, increasing the number of threads was shown to increase performance the most when compared to a proportional change in cores or warps. Another interesting finding was the raster and ROP cache having no impact on performance. These components provided little to no benefit and increased the relative area usage of the fixed function hardware units by 13% and 8% for LUTs and logic registers respectively for a single core configuration. A similar conclusion was made for adding more raster and ROP slices, where more area is used and no performance is gained. Finally, the optimal tile size for area and performance was found to be 16x16 for the Vortex microarchitecture taking into consideration how tile based rendering is implemented within Vortex's raster unit. While a good number of Vortex configurations were tested here, many more can be implemented and tested because of the modularity built into the design.

In addition to Vortex's modular design, a method for extending the Vortex GPU with fixed function hardware units was described, opening the door for further extensions to be implemented and explored for graphics architecture research. These extensions could include hardware accelerator units for popular graphics rendering trends such as ray-tracing and deep learning enhanced super sampling. The raster unit and ROP unit demonstrate how to communicate between fixed function hardware and the programmable cores inside Vortex, all within an open-source codebase.

5.1 Future Work

A necessary addition to this effort would certainly be energy analysis and a more comprehensive area comparison. This work was limited by the availability of high capacity FPGAs able to implement the Vortex GPU. Intel provides access to high capacity FPGAs in their cloud servers to students which would be an ideal place to start. Additionally, this work used Verilator to simulate Vortex with reasonable accuracy and speed but an FPGA implementation would be far more accurate and orders of magnitude faster to complete benchmarks.

Another area for future work would be to explore hardware accelerated ray-tracing on Vortex. Real time ray-tracing has only been recently realized on commercial GPUs and is an ever growing market due to the photo realism achievable with ray traced lighting. Using Vortex as a platform to experiment with various ray-tracing algorithms in hardware would provide useful insight for future GPU architectures.

Finally, a longer term goal for Vortex would be to implement the entire graphics pipeline on the GPU. Relieving the host CPU of vertex processing would allow Vortex to more closely resemble modern GPUs in industry and would not increase hardware requirements significantly since the vertex shader runs on programmable cores just

like the fragment shader. An addition such as this would allow Vortex to be a more useful research tool for desktop and server level GPU architecture research.

BIBLIOGRAPHY

[1] ARM. Tile-based rendering.

[2] A. E. Beasley, C. T. Clarke, and R. J. Watson. An opengl compliant hardware implementation of a graphic processing unit using field programmable gate array–system on chip technology. *ACM Trans. Reconfigurable Technol. Syst.*, 14(1), sep 2020.

[3] B. Ekbote, V. Hire, P. Mahajan, and J. Sisodia. Blockchain based remittances and mining using cuda. In *2017 International Conference On Smart Technologies For Smart Nation (SmartTechCon)*, pages 908–911, 2017.

[4] T. K. V. W. Group. Opengl es specification, May 2022.

[5] T. K. V. W. Group. Vulkan® 1.3.217 - a specification (with all registered vulkan extensions), Jun 2022.

[6] Imagination. Powervr sdkn. `https://developer.imaginationtech.com/powervr-sdk/`.

[7] R.-V. International. History of risc-v, Oct 2020.

[8] N. Lopes and B. Ribeiro. Gpumlib: An efficient open-source gpu machine learning library. *International Journal of Computer Information Systems and Industrial Management Applications*, 3:355–362, 2011.

[9] NVidia. Nvidia's next generation cuda compute architecture. *NVidia, Santa Clara, Calif, USA*, 2009.

[10] NVidia. Nvidia ampere ga102 gpu architecture. *NVidia, Santa Clara, Calif, USA*, 2021.

[11] A. Overvoorde. Shader modules.

[12] D. Patterson, J. L. Hennessy, J. Nickolls, and D. Kirk. *Appendix C: Graphics and Computing GPUs*. Elsevier, 1 edition, 2018.

[13] L. Seiler, D. Carmean, E. Sprangle, T. Forsyth, M. Abrash, P. Dubey, S. Junkins, A. Lake, J. Sugerman, R. Cavin, et al. Larrabee: a many-core x86 architecture for visual computing. *ACM Transactions on Graphics (TOG)*, 27(3):1–15, 2008.

[14] W. Snyder. Welcome to verilator.

[15] T. Tamasi. *Nvision 2008*.

[16] Q. Technologies. The rise of mobile gaming on android: Qualcomm snapdragon technology leadership. *Qualcomm Technologies, San Diego, Calif, USA*, 2014.

[17] B. Tine. Vortex graphics taskforce, Apr 2022.

[18] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and K. Hyesoon. Vortex: Extending the risc-v isa for gpgpu and 3d-graphics. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, page 754–766, New York, NY, USA, 2021. Association for Computing Machinery.

[19] J. d. Vries. Hello triangle, Jun 2014.

[20] A. Waterman, K. Asanovic, S. Inc, and U. o. C. B. CS Division, EECS Department. The risc-v instruction set manual, volume: I user-level isa, document version 2.2, May 2017.

[21] O. Wiki. History of programmability — opengl wiki,, 2015. [Online; accessed 19-May-2022].

[22] Y. Zhou, X. Jin, and T. Xiang. RISC-V Graphics Rendering Instruction Set Extensions for Embedded AI Chips Implementation. In *Proceedings of the 2020 2nd International Conference on Big Data Engineering and Technology*, BDET 2020, pages 85–88. Association for Computing Machinery.

APPENDICES

Appendix A

RESULTS FOR SOFTWARE VS HARDWARE RENDERING (CONTINUED)

The following figures contain the remaining benchmark results for the Software vs Hardware Rendering section. The included benchmarks are Vase, Skybox, Coverflow, Evilskull, and Polybump.

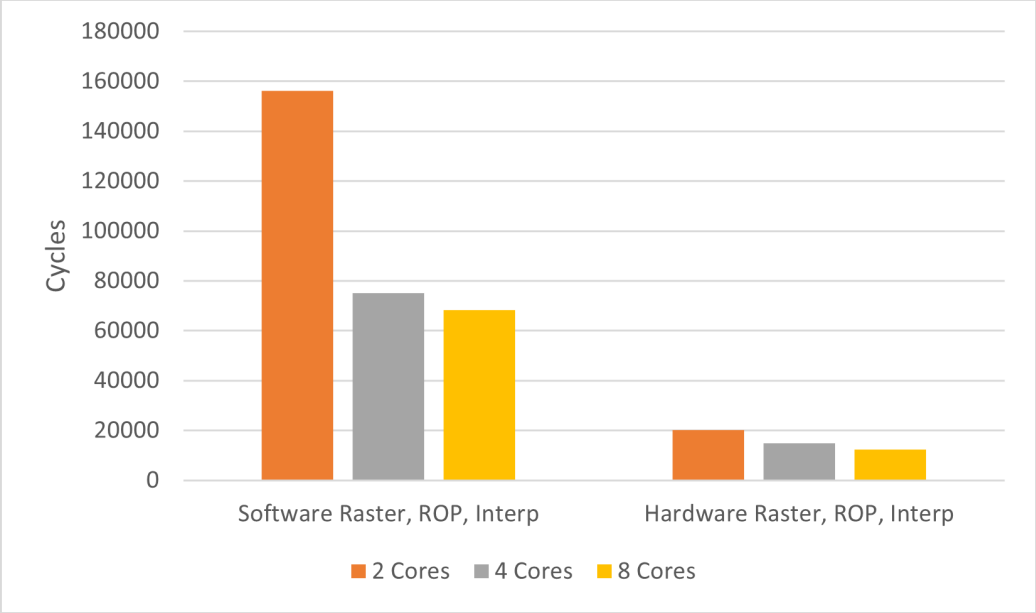


Figure A.1: Total number of clock cycles to complete Vase benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.8 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.

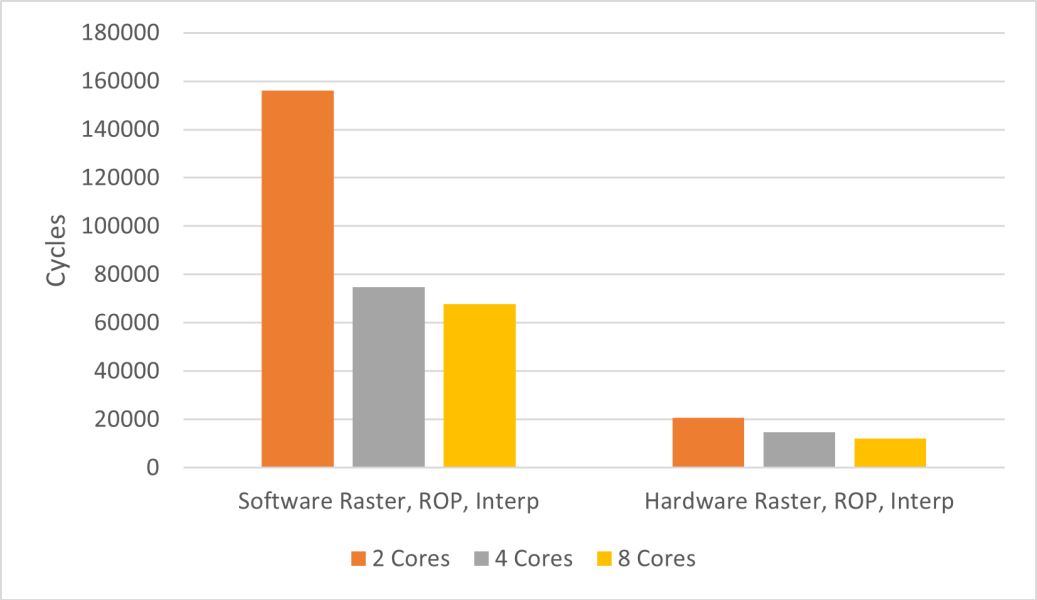


Figure A.2: Total number of clock cycles to complete Skybox benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5.1 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.

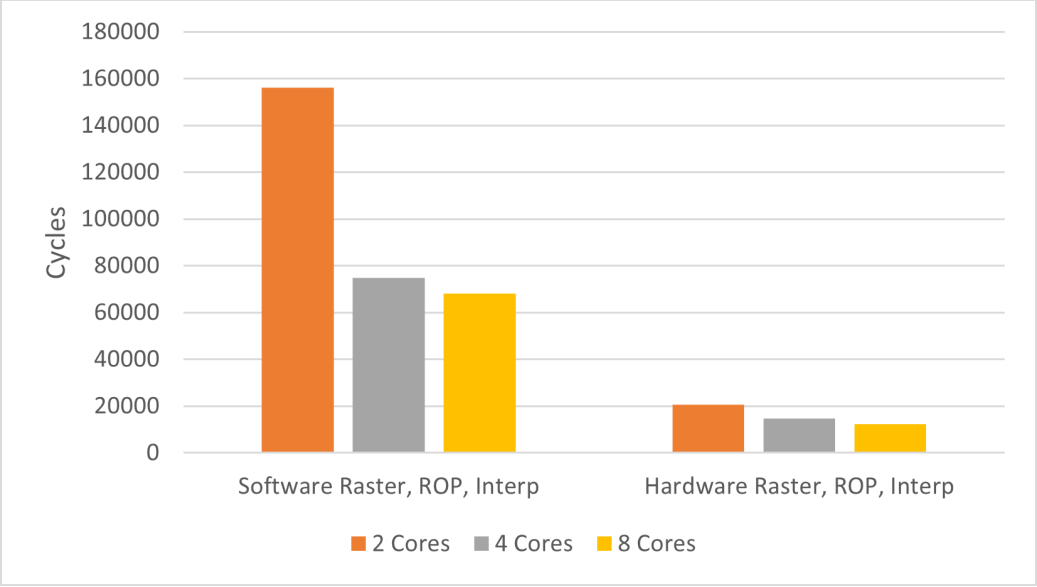


Figure A.3: Total number of clock cycles to complete Coverflow benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5.1 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.

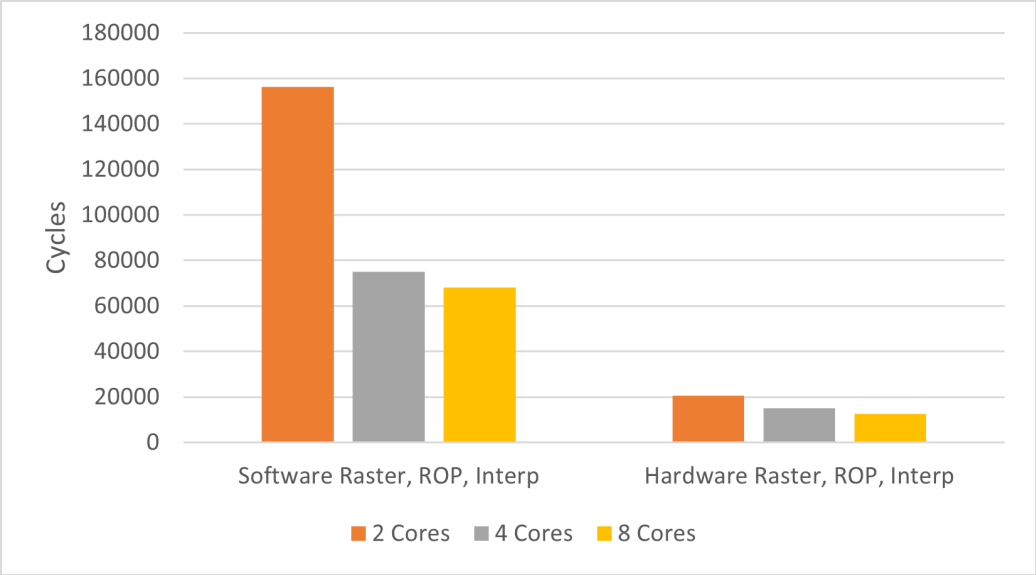


Figure A.4: Total number of clock cycles to complete Evilskull benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.

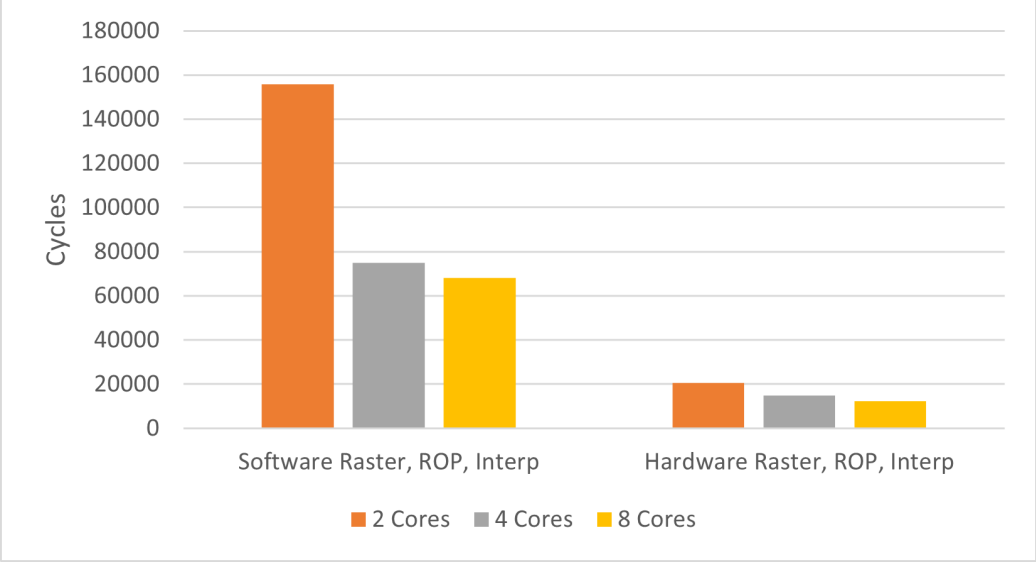


Figure A.5: Total number of clock cycles to complete Polybump benchmark vs number of cores for both software rendering and hardware rendering methods. Hardware rendering completes benchmark 5 to 7.6 times faster than software rendering. Results for one core configuration omitted due to errors in the simulation results.